ML & AI Interview Notes

Applied Scientist — Amazon & Microsoft

Complete study guide covering Machine Learning, Deep Learning, Statistics, System Design, Coding, and Top 100 Interview Questions

---

#	Topic	Page
1	Machine Learning Fundamentals	3
2	Core ML Algorithms	10
3	Model Evaluation & Validation	18
4	Deep Learning & Neural Networks	22
5	Transformers, LLMs & Generative AI	28
6	Statistics & Probability	36
7	A/B Testing & Causal Inference	41
8	Recommendation Systems & NLP	45
9	ML System Design	49
10	Coding Patterns for ML Interviews	53
11	Top 100 Interview Questions & Answers	58

Chapter 1: Machine Learning Fundamentals

This chapter covers the foundations every ML scientist must know cold — the concepts that appear in almost every interview round, from bias-variance to optimization.

1.1 What Is Machine Learning?

Machine learning is teaching a computer to make decisions from data, instead of writing explicit rules. Three main types:

Type	What the model learns from	Real-world example
Supervised	Labeled data (input → known output)	Gmail spam filter: emails labeled spam/not-spam
Unsupervised	Unlabeled data (find structure)	Spotify groups similar songs into playlists automatically
Reinforcement	Rewards/penalties from environment	AlphaGo learns to play Go by playing millions of games

1.2 The Bias-Variance Tradeoff

This is the single most important concept in ML. Every model makes errors. We can decompose any model's error into three parts:

```
Total Error = Bias² + Variance + Irreducible Noise

Bias²    = How far the model's average prediction is from the truth
           (systematic, "wrong assumptions")

Variance = How much the model's prediction changes with different training data
           (sensitivity to data fluctuations)

Noise    = Random noise in the data itself — cannot be reduced
```

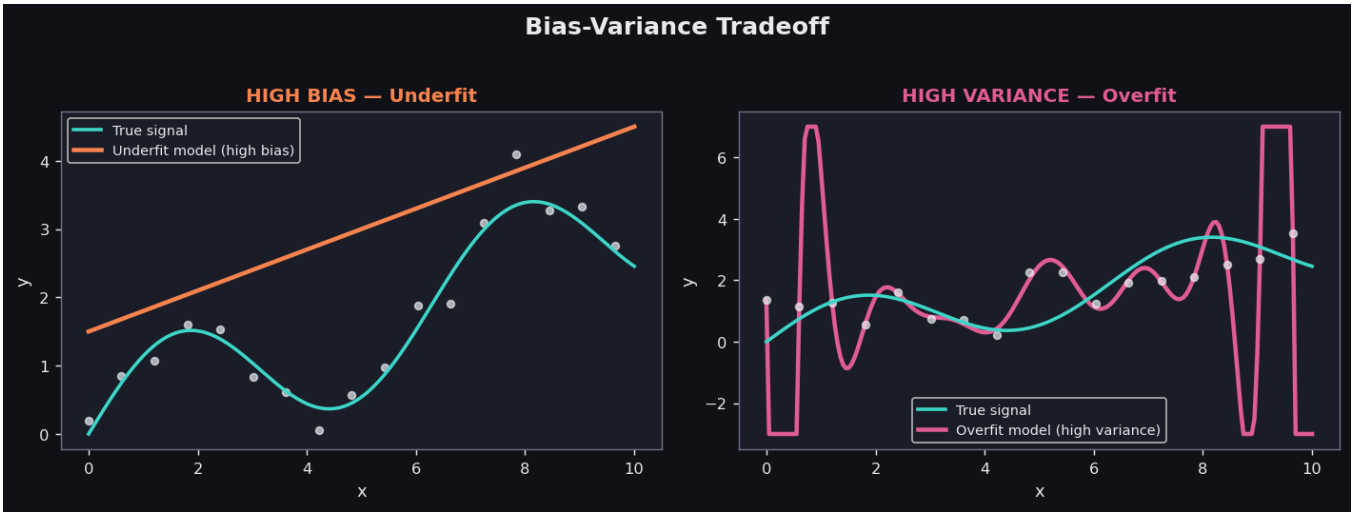

Figure 1.1 — Left: High bias (underfit) — straight line misses the sine curve. Right: High variance (overfit) — the model memorizes noise instead of learning the pattern.

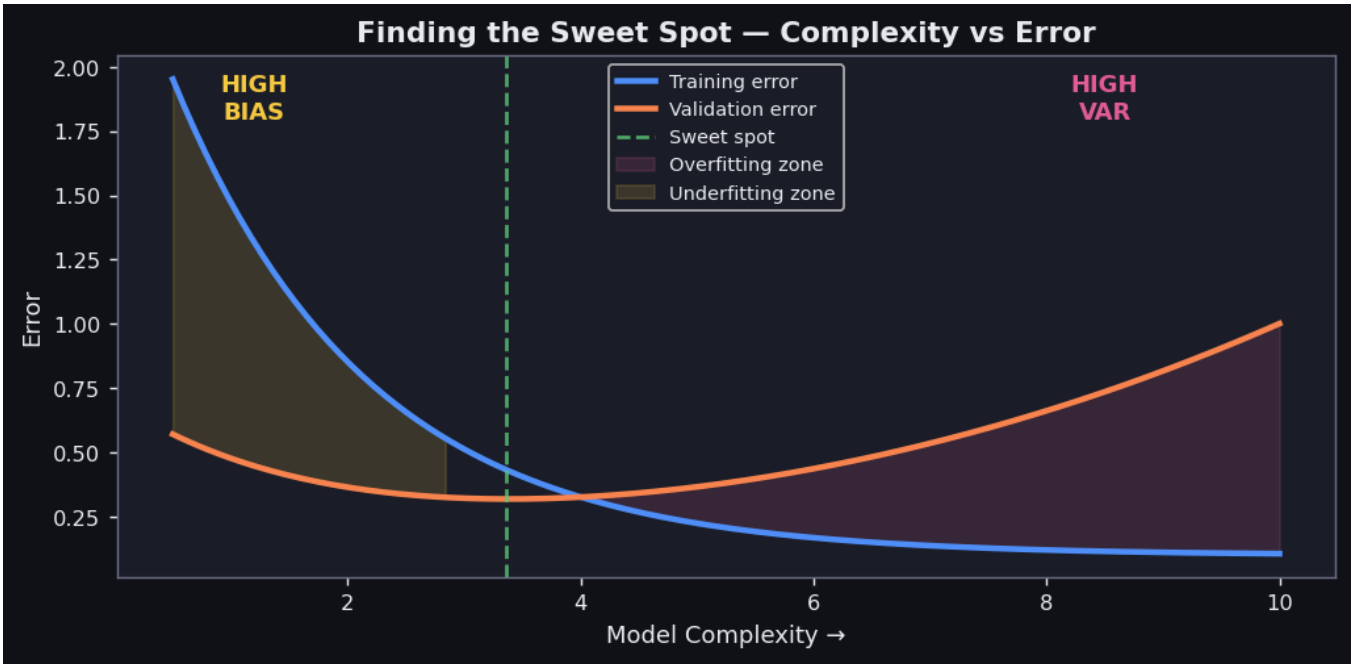

Figure 1.2 — The sweet spot: as model complexity grows, training error keeps falling but validation error has a U-shape. Too simple = underfit. Too complex = overfit.

Real example: Predicting house prices

Situation	Bias	Variance	What happens	Fix
Linear model on complex data	High	Low	Always misses by £50k regardless of data size	Add more features / use tree model
100-degree polynomial	Low	High	Trains perfectly but fails on new houses	Regularize (Ridge/Lasso)
Gradient Boosted Tree, tuned	Low	Low	Good generalization	— sweet spot —

■ Quick diagnostic: If train error and val error are BOTH high → high bias (underfit). If train error is low but val error is high → high variance (overfit).

1.3 Regularization — Preventing Overfitting

Regularization adds a penalty on large weights to prevent the model from memorizing training data. Think of it as telling the model: "find a good fit, but keep things simple."

```
Standard Loss:    Minimize  L(w) = MSE(w)

Ridge (L2):       Minimize  L(w) + λ · Σ w■²
                  → Shrinks ALL weights small, but never to exactly zero

Lasso (L1):       Minimize  L(w) + λ · Σ |w■|
                  → Sets SOME weights to exactly zero (feature selection!)

Elastic Net:      Minimize  L(w) + λ■·Σ|w■| + λ■·Σw■²
                  → Mix of both (use when many correlated features)
```

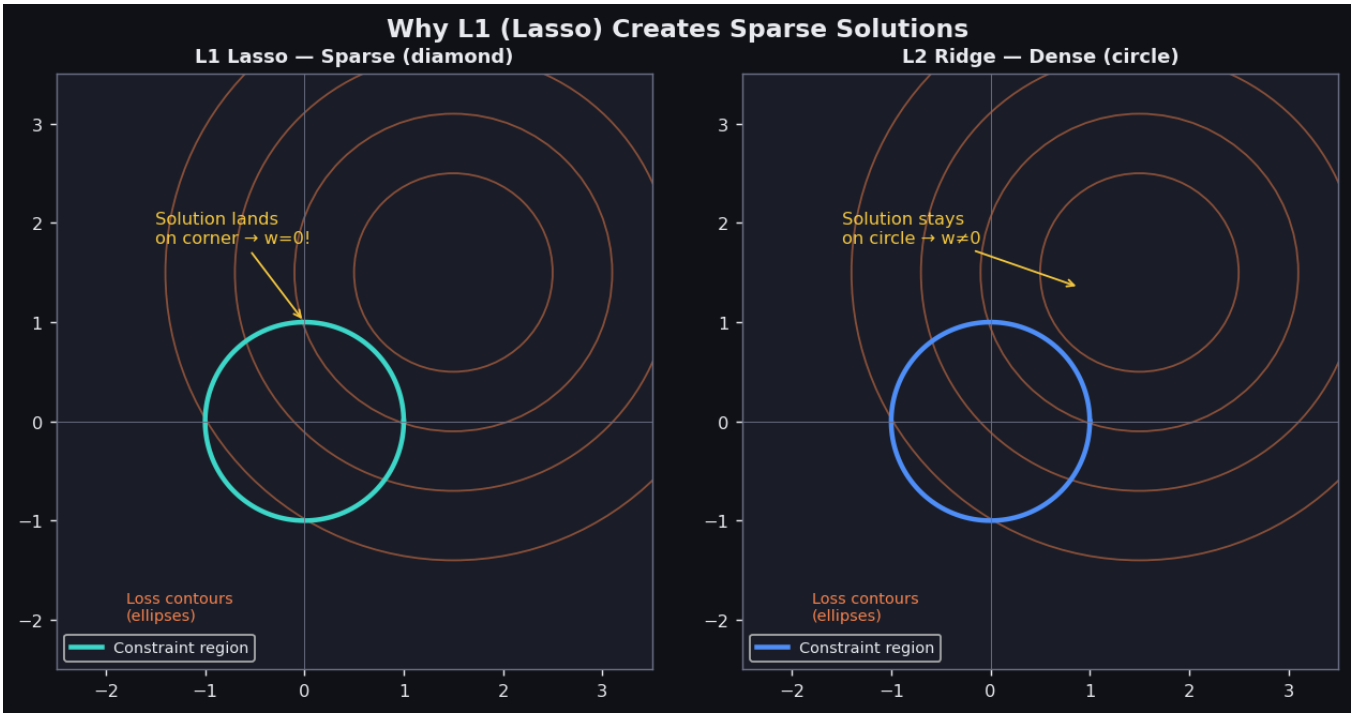

Figure 1.3 — Geometry of L1 vs L2. The loss function's contour ellipses hit the L1 constraint at a corner (forcing w=0), but never hit the L2 circle at a corner.

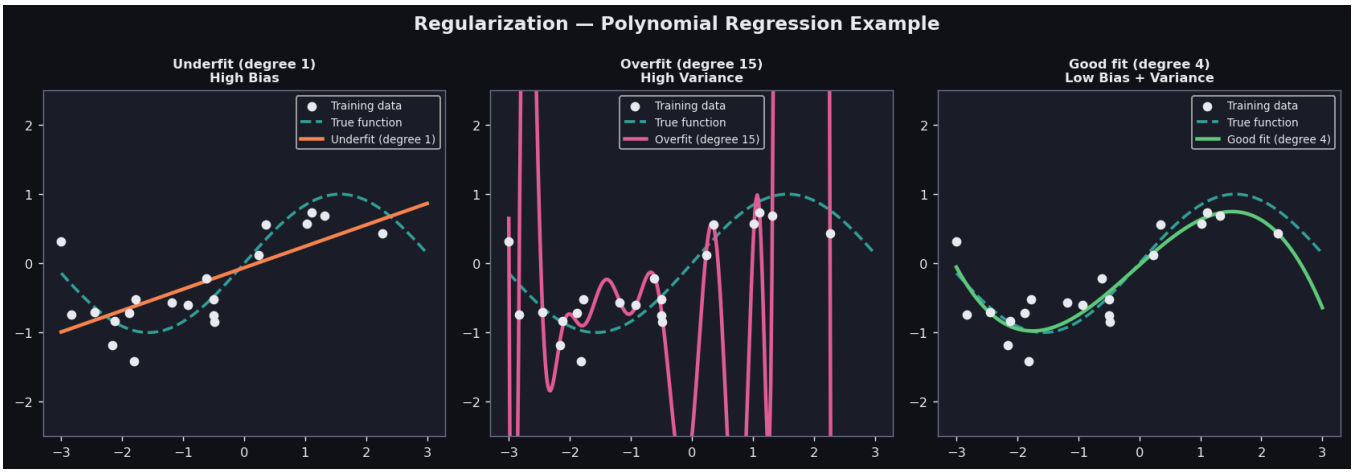

Figure 1.4 — Polynomial regression example: degree 1 underfit, degree 15 overfit, degree 4 just right.

Method	When to use it	Sparsity?	Handles correlated features?
L2 Ridge	All features probably matter; reduce their scale	No	Yes — groups them together
L1 Lasso	100 features but only 10 are relevant	Yes — selects features	No — picks one, drops rest
Elastic Net	Many features, some correlated (e.g., genomics)	Partial	Yes

1.4 Optimization — How Models Learn

Training a model means finding weights that minimize the loss function. We do this with gradient descent — take a small step downhill on the loss surface at each iteration.

```
Core update rule:   w ← w - η · ∇L(w)

  η (eta) = learning rate — how big a step to take
  ∇L(w)  = gradient = direction of steepest ASCENT → we go opposite

Batch GD:      use ALL training data per update → smooth but slow
SGD:           use 1 sample per update → noisy but fast, escapes local minima
Mini-batch GD: use 32–512 samples → best of both worlds (standard in DL)
```

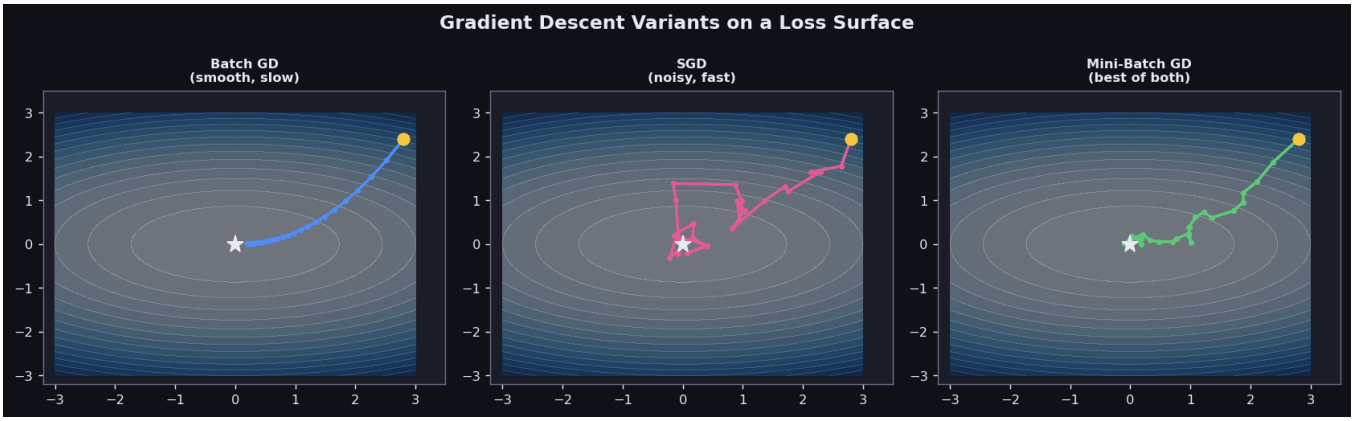

Figure 1.5 — Three gradient descent variants on a 2D loss surface. Yellow star = start. White star = minimum. SGD is noisier but can escape local minima.

Optimizers compared (most important for interviews):

Optimizer	Idea	Best for	Key hyperparameter
SGD + Momentum	Accumulate velocity in gradient direction	CNNs when carefully tuned	lr, momentum=0.9
Adam	Adaptive lr per weight + momentum	NLP, general DL	lr=1e-3, β1=0.9, β2=0.999
AdamW	Adam + proper weight decay (decoupled)	Transformers, LLMs	lr=1e-4, wd=0.01
RMSProp	Adaptive lr, no momentum	RNNs	lr, decay=0.9

■ Why AdamW for transformers? Standard Adam applies weight decay to ALL parameters including adaptive lr scaling — this is wrong. AdamW correctly decouples weight decay.

Learning rate schedules (critical for LLM training):

Schedule	What it does	Used in
Warmup + cosine decay	Linear ramp-up for N steps, then cosine decrease	GPT-4, LLaMA, BERT
Step decay	Multiply lr by 0.1 every N epochs	Classic CNNs
Cyclic LR	Oscillates between min and max lr	ResNet training tricks
One-cycle	Short fast warmup, long cosine down	fast.ai style training

Chapter 2: Core ML Algorithms

Deep knowledge of these algorithms — not just "how to call them" but WHY they work, when they fail, and how they compare — is what separates Applied Scientists from regular engineers.

2.1 Linear & Logistic Regression

Linear Regression: Fit the best straight line (or hyperplane) through data. Used for predicting continuous values (house price, temperature).

```
Prediction:    ■ = w■x■ + w■x■ + ... + w■x■ + b  =  Xw + b

Loss (MSE):    L = (1/n) Σ (y■ - ■■)²

Closed form:   w* = (X■X)■¹ X■y    ← exact, but O(d³) — expensive for many features
In practice:   use gradient descent (SGD) — scales to millions of features
```

Logistic Regression: Despite the name, it's a **classification** model. It adds a sigmoid function to squash the output between 0 and 1 (a probability).

```
Sigmoid:       σ(z) = 1 / (1 + e^(-z))    output ∈ (0, 1)

Prediction:    P(y=1|x) = σ(w■x + b)

Loss (BCE):    L = -(1/n) Σ [y■ log(■■) + (1-y■) log(1-■■)]

Gradient:      ∂L/∂w = (1/n) X■(■ - y)    ← beautifully clean form
```

```python
# Logistic Regression from scratch in 15 lines
import numpy as np

def sigmoid(z): return 1 / (1 + np.exp(-z))

def train(X, y, lr=0.01, epochs=1000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        y_hat = sigmoid(X @ w + b)          # forward pass
        dw = X.T @ (y_hat - y) / n          # gradient w.r.t. weights
        db = (y_hat - y).mean()              # gradient w.r.t. bias
        w -= lr * dw;  b -= lr * db         # gradient descent step
    return w, b

# In sklearn:
# LogisticRegression(C=1.0, penalty="l2", solver="lbfgs")
```

2.2 Decision Trees

A decision tree asks a sequence of yes/no questions about features to arrive at a prediction. Like a flowchart. Highly interpretable — you can print the logic and show a business person.

```
Split criterion (Classification):

  Gini Index:   G = 1 - Σ p■²
  Entropy (IG): H = -Σ p■ log■(p■)
  Info Gain:    IG = H(parent) - weighted_avg H(children)

The tree picks the feature+threshold that maximizes Information Gain at each node.

Split criterion (Regression):
  Minimize variance reduction: Var(parent) - weighted_avg Var(children)
```

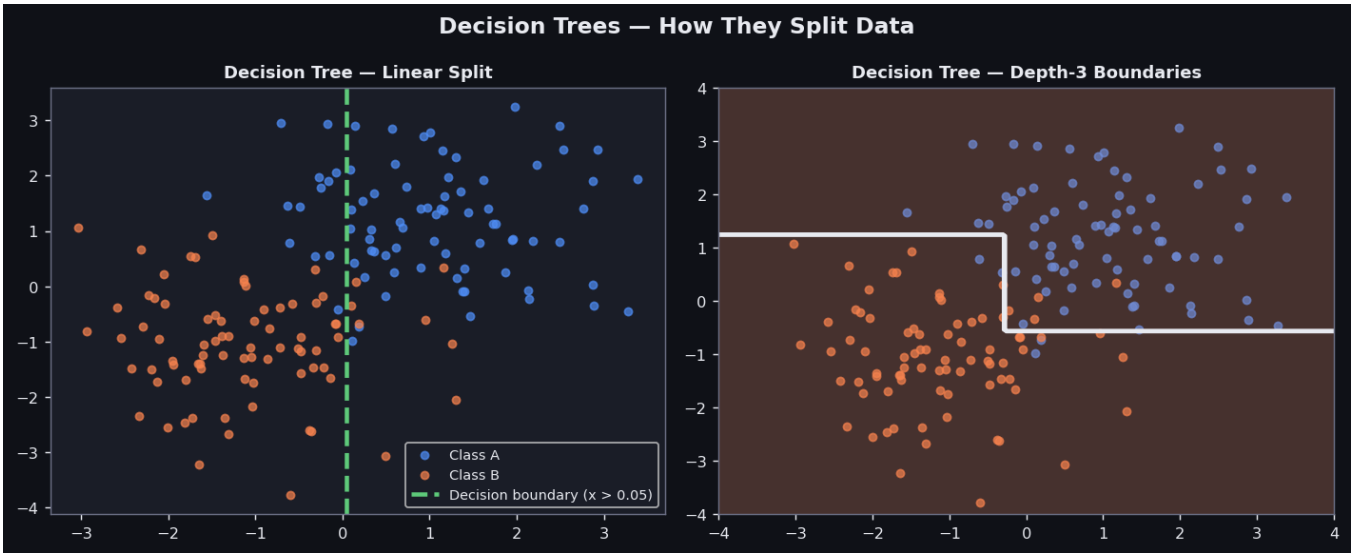

Figure 2.1 — Left: a single linear split. Right: a depth-3 tree creates multiple axis-aligned boundaries, approximating complex shapes.

■■ Decision trees overfit badly with no depth limit. Always use max_depth, min_samples_leaf, or pruning. Or better yet — use Random Forest or Gradient Boosting.

2.3 Random Forest vs Gradient Boosting — The Big Comparison

These are the two most powerful "classical" ML methods. In tabular data competitions, one of these almost always wins. Understanding WHY helps you choose the right one.

	Random Forest	Gradient Boosting (XGBoost/LightGBM)
Idea	Train 100s of trees in PARALLEL on random subsets	Train trees SEQUENTIALLY, each fixing previous errors
Reduces	Variance (ensemble averaging)	Bias (iterative residual fitting)
Speed (train)	Fast (parallelizable)	Slower (sequential)
Speed (predict)	Fast	Fast
Overfitting risk	Low (bagging acts as regularizer)	Higher — needs careful tuning
Best when	Quick baseline, noisy data, interpretability OK	Competition, maximize accuracy on clean data

	Random Forest	Gradient Boosting (XGBoost/LightGBM)
Key hyperparams	n_estimators, max_depth, max_features	learning_rate, n_estimators, max_depth, early stopping
Industry use	Feature importance at Amazon	CTR prediction, ranking at Microsoft/Bing

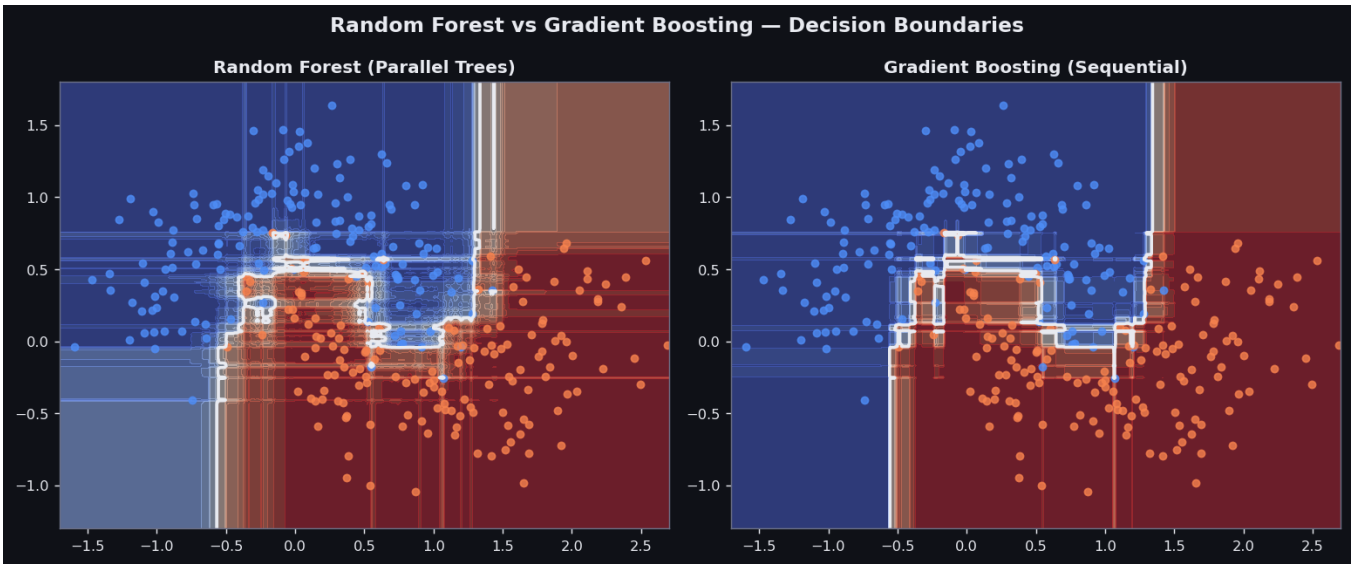

Figure 2.2 — Decision boundaries on the "moons" dataset. Both achieve similar quality; Gradient Boosting is often slightly smoother.

How Gradient Boosting works (conceptually):

Round	Tree trains on...	What it learns
Tree 1	Original labels y	Rough approximation
Tree 2	Residuals: (y - Tree1 prediction)	Fixes Tree 1's biggest mistakes
Tree 3	Residuals from Tree 1 + Tree 2	Fixes remaining errors
...	...	...
Final	Sum of all trees × learning rate	Highly accurate ensemble

```
Gradient Boosting update:

  F_m(x) = F_{m-1}(x) + η · h_m(x)

  where h_m fits the NEGATIVE GRADIENT (pseudo-residuals):
  r■ = -∂L/∂F(x■) evaluated at F = F_{m-1}

  For MSE loss: r■ = y■ - F_{m-1}(x■)   (actual residuals)
  For Log loss: r■ = y■ - σ(F_{m-1}(x■)) (probability residuals)
```

```python
# XGBoost — production-ready gradient boosting
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=500,     # max number of trees
    max_depth=6,          # depth per tree
    learning_rate=0.05,   # η — smaller = more trees needed but better
    subsample=0.8,        # fraction of rows per tree (reduces variance)
    colsample_bytree=0.8, # fraction of cols per tree (reduces variance)
    reg_lambda=1.0,       # L2 regularization on leaf weights
    eval_metric="auc",
    early_stopping_rounds=50  # stop if val AUC does not improve
)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=100)
```

2.4 Support Vector Machines (SVM)

SVM finds the maximum-margin hyperplane — the decision boundary that is farthest from the nearest training points of each class. Only the "support vectors" (closest points) matter.

```
Find w, b to:   max margin = 2/||w||     (maximize gap between classes)
Subject to:     y■(w■x■ + b) ≥ 1       for all training points

Equivalent:     min  ½||w||²             (minimize weight magnitude)

Soft margin (real data has noise):
  min  ½||w||² + C · Σ ξ■
  ξ■ ≥ 0 = slack variables (allow some misclassification)
  Large C → small margin, low training error (risk overfit)
  Small C → large margin, allows errors (risk underfit)

Kernel trick: K(x, z) = ϕ(x)■ϕ(z)  — no need to compute ϕ explicitly!
  RBF kernel: K(x,z) = exp(-γ||x-z||²)  — infinite dimensional space
```

Kernel	Formula	When to use
Linear	x■z	Text classification, high-dimensional sparse data
RBF (Radial Basis)	exp(-γ||x-z||²)	General purpose, non-linear boundaries
Polynomial	(γx■z + r)^d	Image features, when degree is meaningful

■■ SVMs scale as $O(n^2)$ to $O(n^3)$ with dataset size. Not suitable for >100K samples. For large datasets, use SGDClassifier(loss="hinge") which approximates SVM linearly.

2.5 K-Nearest Neighbors & Naive Bayes

K-Nearest Neighbors (KNN):

Stores all training data. Predicts by majority vote of k nearest neighbors. No "training" — it is a lazy learner.

k=1	Very complex boundary, memorizes training data	High Variance
k=n (all)	Always predicts majority class	High Bias
k=5 to 20	Tuned by cross-validation	Sweet spot

■■ ALWAYS scale features before KNN! Euclidean distance is sensitive to scale. A feature measured in thousands will dominate one measured in ones.

Naive Bayes:

```
P(class|features) ∝ P(class) × P(f■|class) × P(f■|class) × ...

"Naive" = assumes features are INDEPENDENT given the class
          (almost never true, but model still works surprisingly well)

Gaussian NB:    P(f■|class) = Normal distribution (for continuous features)
Multinomial NB: P(f■|class) ∝ word count  (for text — spam filter)
Laplace smooth: add α to counts to avoid P=0 for unseen words
```

2.6 Clustering — Unsupervised Learning

Find natural groupings in data without labels. Amazon uses clustering for customer segmentation; Spotify uses it for music genre discovery.

Figure 2.3 — K-Means converges in 3-4 steps. White stars = centroids. Colors = cluster assignments. K-Means++ places initial centroids far apart for better results.

```
K-Means objective:  minimize WCSS = Σ■ Σ■∈C■ ||x - μ■||²

E-step: assign each point to nearest centroid
M-step: update each centroid to mean of its assigned points
Repeat until convergence

Choosing k:  Elbow method — plot WCSS vs k, look for elbow
             Silhouette score: (b-a)/max(a,b) ∈ [-1, 1], higher = better
```

Algorithm	Key idea	Advantage	Limitation
K-Means	Minimize within-cluster variance	Fast, scalable (O(nkT))	Needs k upfront, assumes spherical clusters
DBSCAN	Cluster by density, noise = outliers	Arbitrary shapes, detects outliers	Poor with varying densities
GMM (EM)	Soft assignments via Gaussian mixture	Probabilistic, handles overlap	Assumes Gaussian shape
Hierarchical	Build tree of merges/splits	Dendrogram shows structure	O(n²) — slow for large data

2.7 Dimensionality Reduction — PCA & Beyond

PCA finds the directions of maximum variance in high-dimensional data and projects data onto a lower-dimensional subspace. Used for visualization, noise reduction, and speeding up downstream

models.

```
Steps:
1. Center data:          X■ = X - mean(X)
2. Covariance matrix:    Σ = X■■X■ / (n-1)
3. Eigendecomposition:   Σ = VΛV■  (V = eigenvectors, Λ = eigenvalues)
4. Project:              Z = X■ · V_k  (keep top k components)

Equivalently via SVD (more numerically stable):
  X = UΣV■  →  principal components = right singular vectors (V)
```

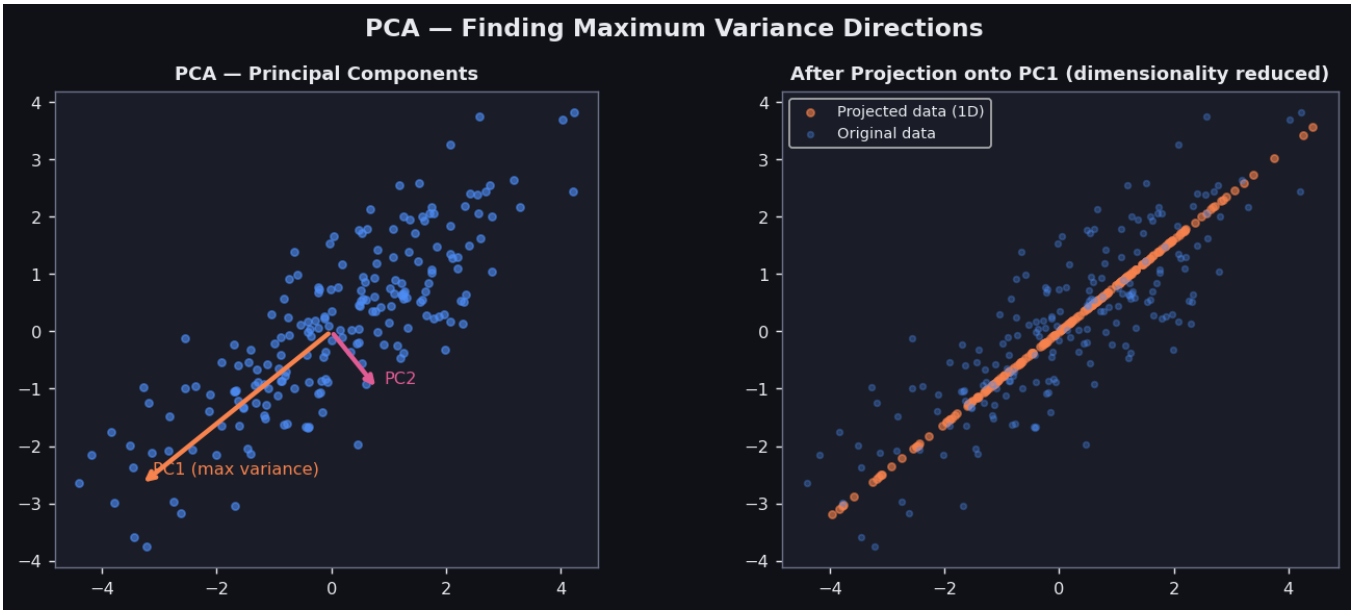

Figure 2.4 — Left: the two principal components (orange = PC1, pink = PC2) on 2D data. PC1 captures most variance. Right: projecting onto PC1 reduces dimensionality from 2D → 1D.

Method	What it preserves	Speed	Use case
PCA	Global variance (linear)	Fast	Preprocessing, noise reduction, visualization
t-SNE	Local neighborhood structure	Slow $O(n^2)$	Visualization only (2D/3D)
UMAP	Local + some global structure	Fast $O(n \log n)$	Visualization + downstream tasks
Autoencoder	Non-linear structure	Slow (training)	Complex manifolds, reconstruction

■■ t-SNE and UMAP distances are NOT meaningful — only topology matters. You cannot say "cluster A is 3x farther from cluster B than C." Cluster sizes and inter-cluster distances are meaningless artifacts.

Chapter 3: Model Evaluation & Validation

Knowing how to properly evaluate a model is just as important as building it. Wrong metrics lead to wrong decisions — even models that look great on paper can fail badly in production.

3.1 Classification Metrics

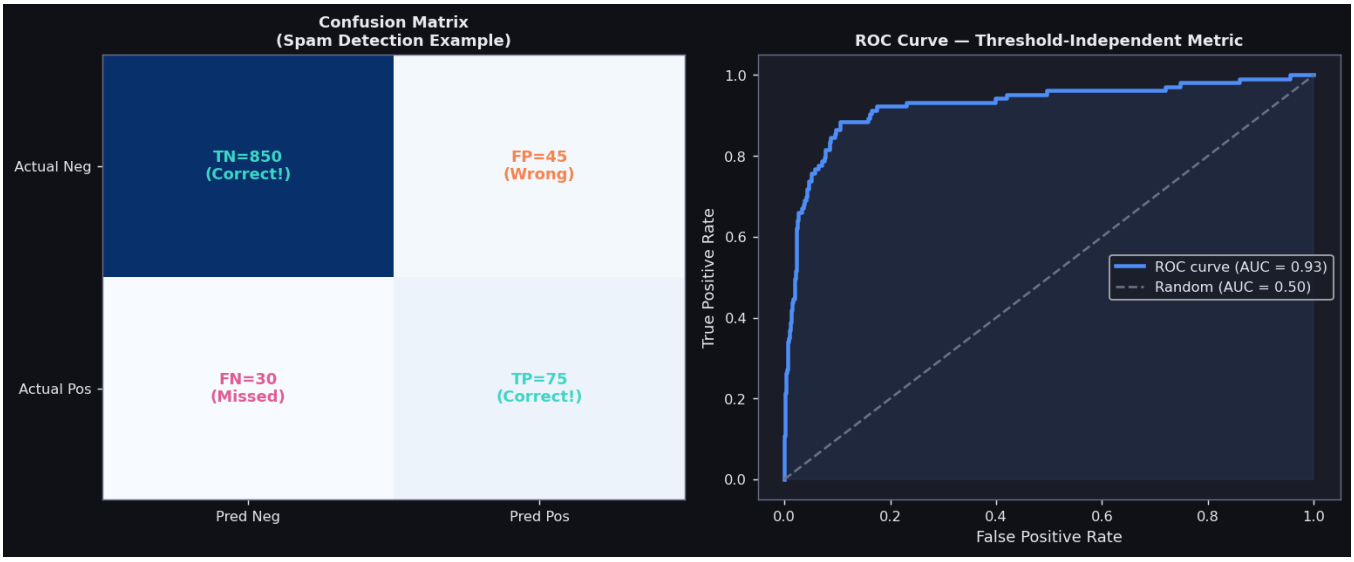

Figure 3.1 — Left: Confusion matrix for a spam detector. Right: ROC curve (threshold-independent ranking quality).

```
From the confusion matrix:

  Precision  = TP / (TP + FP)    "Of all I called spam, how many were actually spam?"
  Recall     = TP / (TP + FN)    "Of all actual spam, how much did I catch?"
  F1 Score   = 2 × (P × R) / (P + R)   harmonic mean of precision and recall

  AUC-ROC = P(score_pos > score_neg)  threshold-independent ranking quality
            = area under TPR vs FPR curve
            = 0.5 (random) to 1.0 (perfect)

  AUC-PR = area under Precision vs Recall curve
           USE THIS for imbalanced data (fraud, rare disease)
```

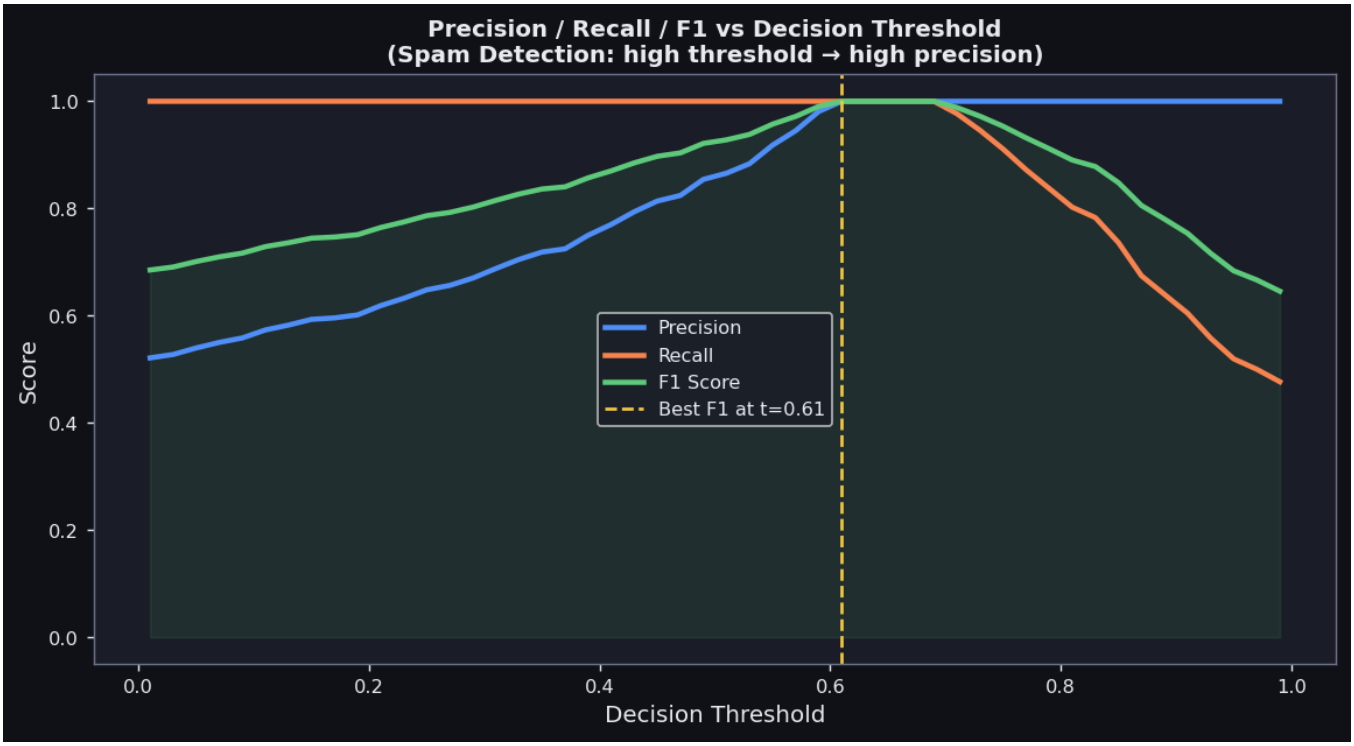

Figure 3.2 — Precision and recall trade off as you move the decision threshold. Choose the threshold that optimizes the business cost function.

Use case	Optimize for	Why
Spam filter	High Precision	Blocking legitimate email (FP) is worse than missing spam (FN)
Cancer screening	High Recall	Missing a cancer (FN) is much worse than a false alarm (FP)
Fraud detection	AUC-PR + Recall	Very imbalanced; catching all fraud matters more than false alerts
Search ranking	NDCG, MRR	Position matters — top results must be most relevant
Product recommender	Precision@K, NDCG@K	Only show top-K items to user

3.2 Cross-Validation — Honest Model Evaluation

Method	How it works	When to use
k-Fold CV	Split into k parts; rotate test fold	Standard (k=5 or 10)
Stratified k-Fold	Maintain class ratios per fold	Imbalanced classification
Leave-One-Out (LOO)	k=n; test on each single point	Very small datasets (<100 samples)
Time Series Split	Only use PAST data to predict future	Any temporal data — NEVER shuffle!
Nested CV	Outer loop: eval; inner loop: hyperparam tuning	Unbiased evaluation when tuning

■■ Data Leakage is the #1 mistake in ML evaluation. NEVER fit your preprocessing (scaler, imputer, encoder) on the full dataset. Fit ONLY on the training fold, then transform train and test. Use sklearn Pipeline to enforce this.

```python
# The right way: use Pipeline to prevent leakage
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

pipe = Pipeline([
    ("scaler", StandardScaler()),    # fit on train fold ONLY
    ("model", LogisticRegression())
])

# cross_val_score handles the fold splitting automatically
scores = cross_val_score(pipe, X, y, cv=5, scoring="roc_auc")
print(f"AUC: {scores.mean():.3f} ± {scores.std():.3f}")
```

Chapter 4: Deep Learning & Neural Networks

Deep learning powers almost every state-of-the-art system at Amazon and Microsoft. From Alexa to Bing to Copilot — understanding how neural networks learn is non-negotiable.

4.1 Neural Networks — The Building Block

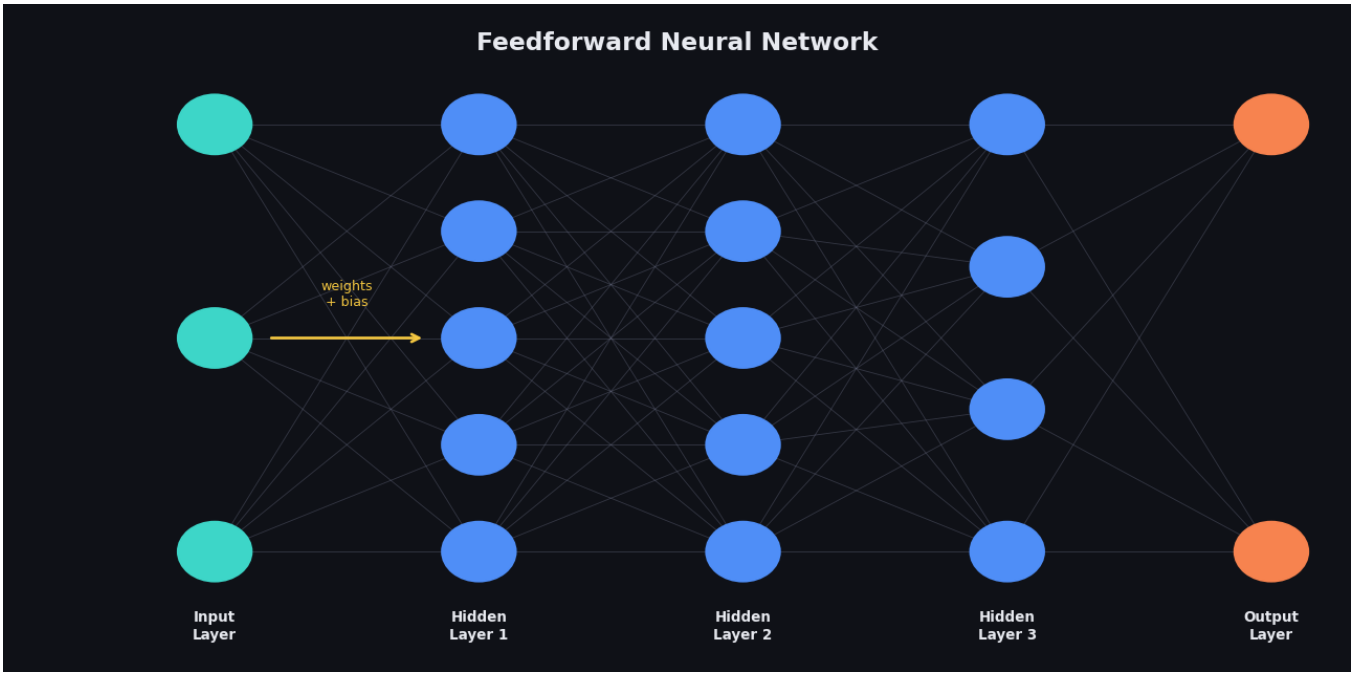

Figure 4.1 — A feedforward network. Each node = a neuron. Each edge = a weight. The network learns these weights during training.

```
One neuron (hidden unit):

  z = w■x■ + w■x■ + ... + w■x■ + b    (linear combination)
  a = activation(z)                     (non-linear transformation)

Why non-linearity? Without it, stacking layers is still just a linear model!
  Linear: W■(W■(W■x)) = (W■W■W■)x = Wx  ← still linear

Forward pass (prediction):
  layer by layer: X → Z■ = XW■+b■ → A■ = relu(Z■) → Z■ → ... → output

Backward pass (learning = backpropagation):
  compute ∂L/∂W using chain rule from output back to inputs
```

4.2 Activation Functions

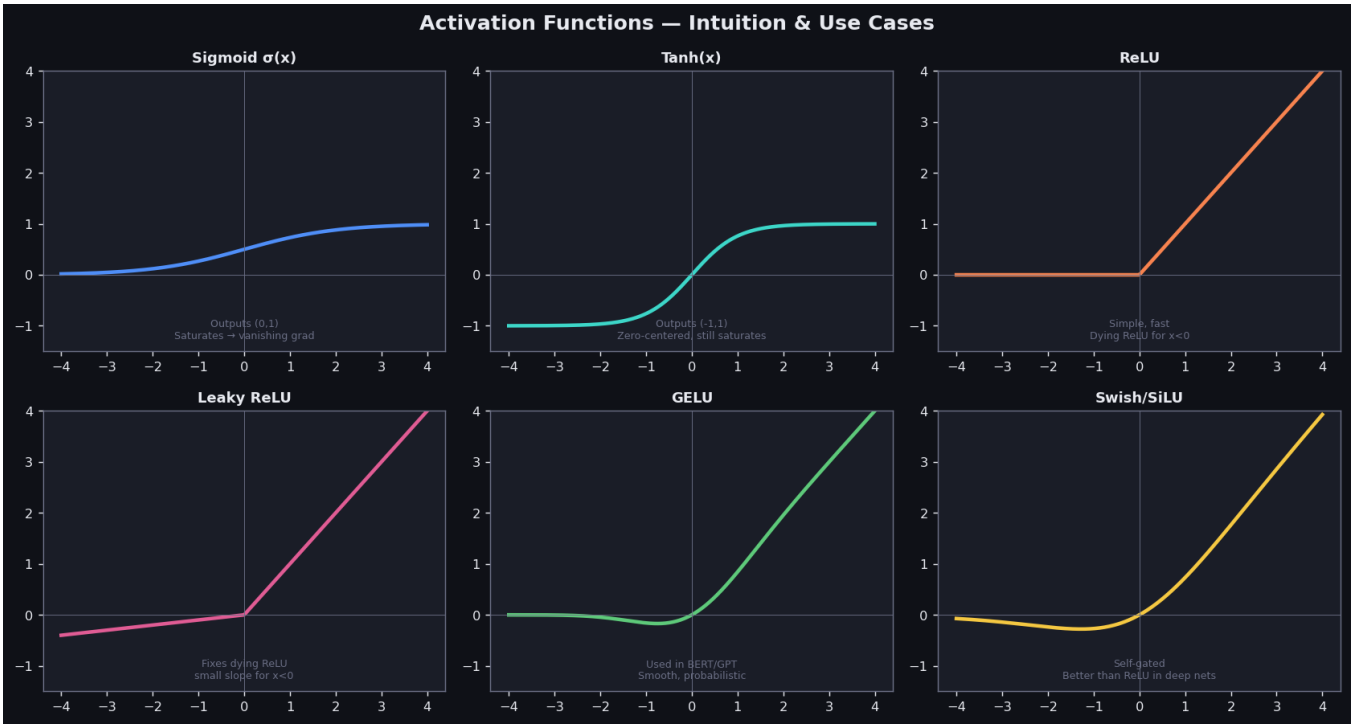

Figure 4.2 — Six activation functions. The key insight: ReLU and its variants are preferred because they do not saturate, preventing vanishing gradients.

Function	Output range	Problem	Use it when...
Sigmoid	(0, 1)	Vanishing gradient for |x|>3	Output layer for binary classification
Tanh	(-1, 1)	Vanishing gradient (less severe)	RNNs (hidden state)
ReLU	[0, ∞)	Dying ReLU (x<0 → dead neurons)	Standard choice for most hidden layers
Leaky ReLU	(-∞, ∞)	α is a hyperparameter	When dying ReLU is a concern
GELU	(-∞, ∞)	More compute than ReLU	Transformers: BERT, GPT
SiLU/Swish	(-∞, ∞)	Non-monotone	LLaMA, modern architectures

4.3 Backpropagation — How Weights Are Updated

Backpropagation is just the chain rule from calculus, applied to a computational graph. It computes how much each weight contributed to the final loss.

```
Chain rule in 1D:   dL/dx = (dL/dy) · (dy/dx)

For a 2-layer network: L → ■ → z■ → a■ → z■ → w■

  ∂L/∂W■ = ∂L/∂■ · ∂■/∂z■ · ∂z■/∂a■ · ∂a■/∂z■ · ∂z■/∂W■

Vanishing gradient problem:
  Sigmoid derivative: σ'(z) = σ(z)(1-σ(z)) ≤ 0.25
  With 10 layers:  gradient ≤ (0.25)^10 ≈ 0.000001  → weights barely move!

Solutions:
  - ReLU: derivative = 1 for z>0, never shrinks gradient
  - Residual connections: ∂L/∂x = ∂L/∂H · (∂F/∂x + 1)  ← the +1 saves us
  - Batch normalization: keeps activations in non-saturating regime
  - Gradient clipping: if ||g|| > threshold → g = g · threshold/||g||
```

4.4 Normalization & Regularization in Deep Learning

Technique	Formula / Idea	Where to use
Batch Norm	Normalize across batch dimension: x■=(x-μ_B)/σ_B, then scale: γx■+β	CNNs — after conv, before activation
Layer Norm	Normalize across feature dimension (per token)	Transformers — avoids batch-size dependency
Dropout	Randomly zero out neurons with probability p during training	Dense layers (p=0.5), transformers (p=0.1)
Weight Decay	L2 penalty on weights during optimization (AdamW)	All deep networks
DropPath	Drop entire residual blocks randomly	Vision Transformers (ViT)

```python
# PyTorch: Build a simple MLP with best practices
import torch
import torch.nn as nn

class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim, dropout=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden),
            nn.LayerNorm(hidden),     # normalize
            nn.GELU(),                # non-linearity
            nn.Dropout(dropout),      # regularize
            nn.Linear(hidden, out_dim)
        )
    def forward(self, x): return self.net(x)

# Training loop with gradient clipping
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
for xb, yb in dataloader:
    loss = loss_fn(model(xb), yb)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    optimizer.zero_grad()
```

Chapter 5: Transformers, LLMs & Generative AI

The transformer architecture (Vaswani et al., 2017) powers BERT, GPT, LLaMA, Gemini, and every modern LLM. This chapter explains it from scratch, in plain language.

5.1 The Transformer Architecture

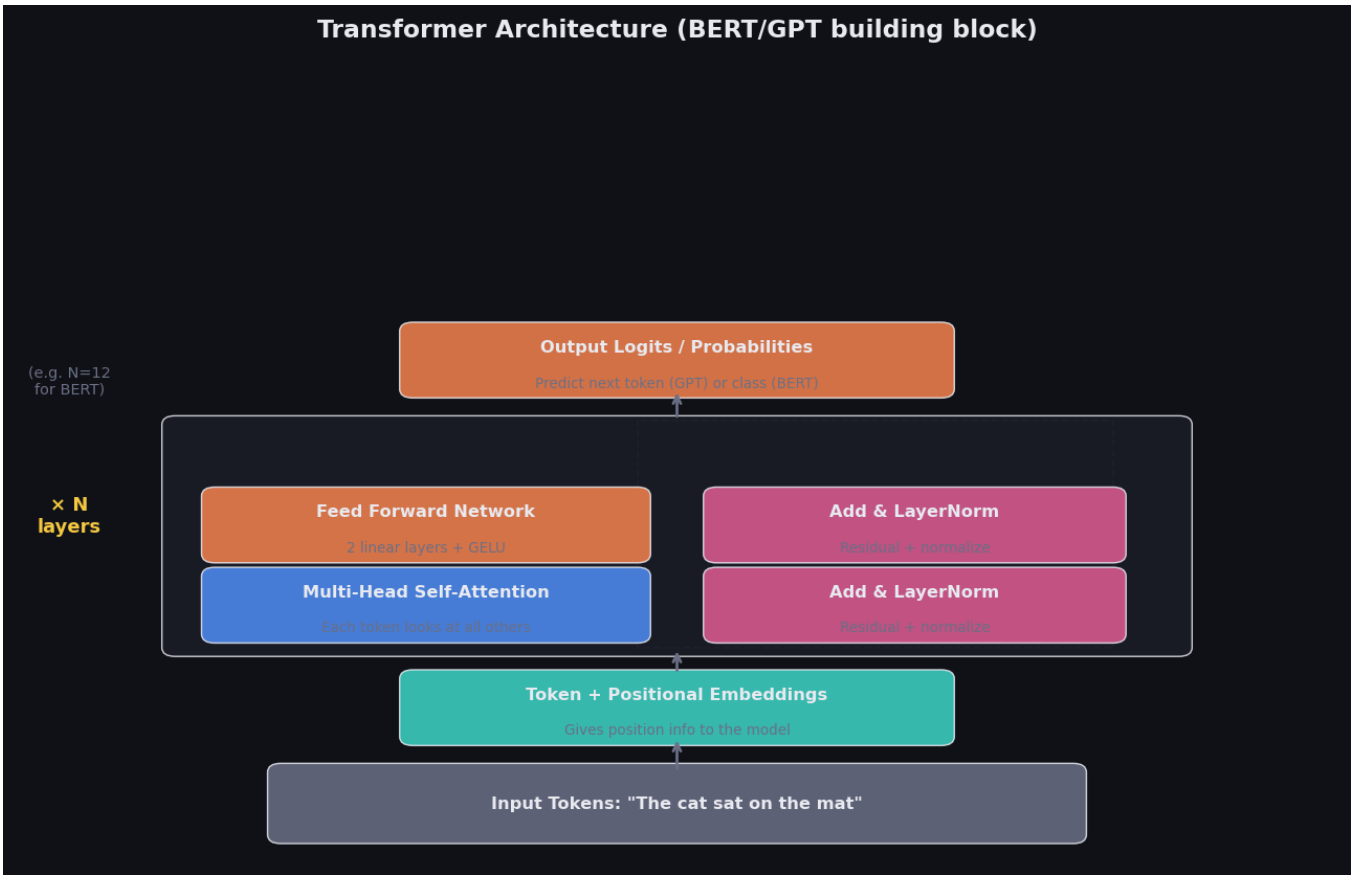

Figure 5.1 — One transformer "block." Stacking N of these gives BERT/GPT. The key insight: skip connections + layer norm make very deep stacks trainable.

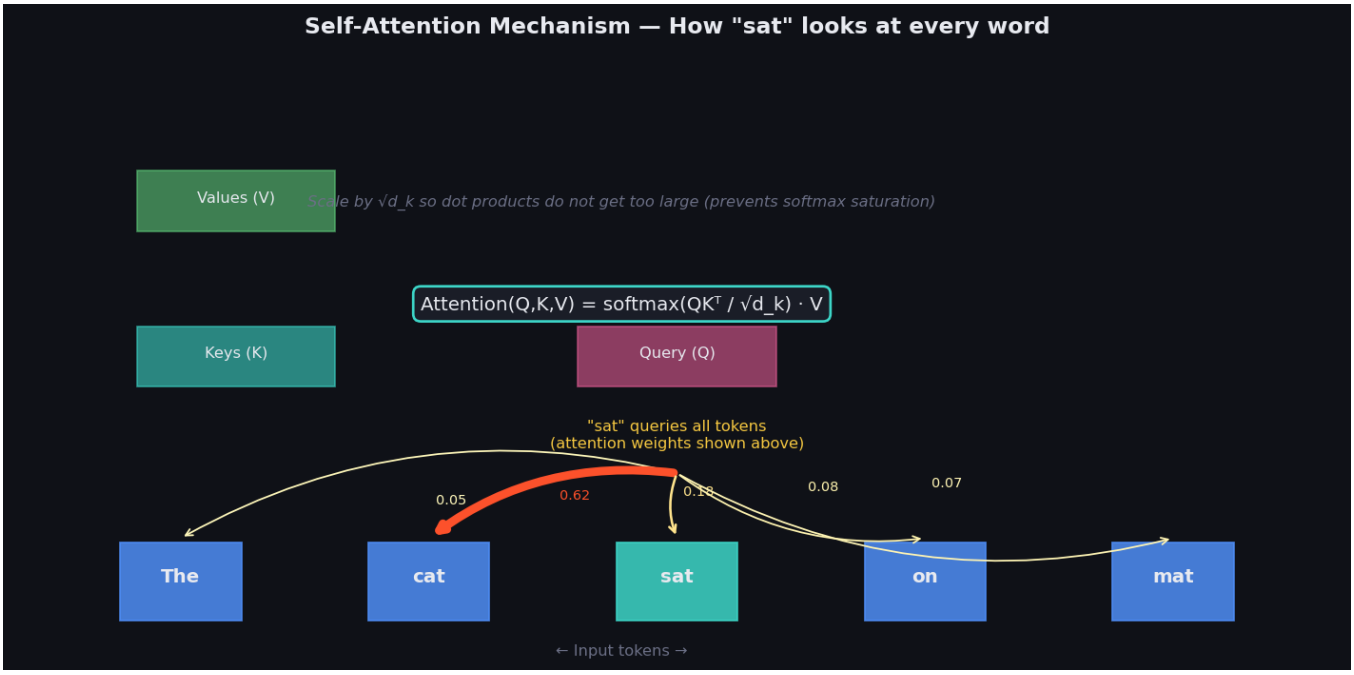

Figure 5.2 — Self-attention. The token "sat" queries all other tokens. Arrow thickness = attention weight. "sat" attends strongly to "cat" (subject-verb relation).

```
Scaled Dot-Product Attention:

  Attention(Q, K, V) = softmax(QK■ / √d_k) · V

  Q = query matrix  (what am I looking for?)
  K = key matrix    (what do I offer?)
  V = value matrix  (what information do I carry?)

  √d_k scaling: prevents dot products from becoming too large,
                which would push softmax into saturation (near-zero gradients)

Multi-Head Attention (h heads):
  head_i = Attention(Q·W■q, K·W■k, V·W■v)
  MHA = Concat(head■, ..., head_h) · W_o
  → Each head can attend to different aspects (syntax, semantics, position)
```

Complexity	Standard Attention	Efficient Alternatives
Time	$O(n^2 \cdot d)$ — quadratic in sequence length!	FlashAttention: same but IO-efficient (GPU-friendly)
Memory	$O(n^2)$ — stores full attention matrix	Sparse attention (Longformer): $O(n \cdot w)$ sliding window
Long sequences	Breaks at ~8K tokens standard	GQA (grouped queries), RoPE (rotary position)

5.2 BERT vs GPT vs T5 — Which Model for Which Task?

Model	Architecture	Pre-training task	Best for
BERT	Encoder-only, bidirectional	Masked LM + Next Sentence Prediction	Classification, NER, QA (extractive)
GPT	Decoder-only, left-to-right	Next token prediction (causal LM)	Text generation, chat, instruction following

Model	Architecture	Pre-training task	Best for
T5	Encoder-Decoder	Span denoising (text-to-text)	Translation, summarization, structured prediction
LLaMA-3	Decoder-only + GQA + RoPE	Next token prediction	Open-source, fine-tuning, research

5.3 Fine-Tuning Large Language Models

You cannot train a 70B parameter model from scratch for every task. Parameter-efficient fine-tuning (PEFT) adapts pre-trained LLMs to new tasks with a tiny fraction of parameters.

Method	Trainable Params	Key Idea	Memory savings
Full Fine-tuning	100% (~billions)	Update all weights	None — most expensive
LoRA	0.1–1%	Add low-rank matrices ΔW=AB alongside frozen W■	3–10×
QLoRA	0.1–1%	LoRA + 4-bit NF4 quantization of base model	10–30×
Prefix Tuning	<0.1%	Prepend learnable tokens to each layer	10×
Adapter	~3%	Insert small bottleneck modules in each layer	3–5×

```
LoRA (Low-Rank Adaptation):

  Standard weight update: W → W + ΔW
  ΔW has d×k parameters — too many!

  LoRA factorizes: ΔW = B·A  where B ∈ ■^(d×r), A ∈ ■^(r×k), r << min(d,k)
  Initialize A ~ N(0, σ²), B = 0  →  starts as identity

  Example: d=4096, k=4096, r=8
    Full: 4096×4096 = 16.7M params
    LoRA: 4096×8 + 8×4096 = 65K params  →  256× fewer!
```

5.4 RLHF — Teaching Models to Be Helpful and Safe

ChatGPT, Claude, and Gemini all use RLHF to align raw LLM output with what humans actually want.

Step	What happens	Why needed
1. SFT	Fine-tune on high-quality human demonstrations	Base model generates gibberish when asked to chat
2. Reward Model	Train model on human preference pairs (A vs B, which is better?)	Need a signal to optimize for helpfulness
3. PPO	Optimize LLM to maximize reward, penalized by KL divergence	KL penalty prevents reward hacking
4. DPO (alternative)	Directly optimize preferences without explicit RM	Simpler, often works as well as PPO

```
PPO objective:

  max_π E[r(x,y)] - β · KL(π || π_SFT)

  r(x,y)    = reward model score (how helpful is this response?)
  π_SFT     = the supervised fine-tuned model
  β         = KL penalty coefficient (prevents moving too far from SFT)

DPO (Direct Preference Optimization):
  Reframes RLHF as supervised learning on preference pairs
  No need for separate reward model — simpler pipeline
```

5.5 Generative Models — VAE, GAN, Diffusion

Model	Key idea	Strength	Weakness
VAE	Encode to latent distribution z~N(μ,σ²), decode back	Smooth latent space, interpretable	Blurry outputs
GAN	Generator vs Discriminator game	Sharp, realistic samples	Mode collapse, hard to train
Diffusion (DDPM)	Learn to reverse a noise-adding process	State-of-the-art image quality + diversity	Slow sampling (1000 steps)
DDIM	Deterministic diffusion with fewer steps	10-50× faster than DDPM	Slightly worse diversity

```
Diffusion forward process (adding noise):
  q(x_t | x_{t-1}) = N(x_t; √(1-β_t) x_{t-1}, β_t I)

Diffusion reverse process (denoising — what the model learns):
  p_θ(x_{t-1} | x_t) = N(x_{t-1}; μ_θ(x_t, t), Σ_θ(x_t, t))

Training objective (predict the noise):
  L = E_t,x■,ε [ ||ε - ε_θ(x_t, t)||² ]

Guidance (text-to-image like DALL-E 3):
  Classifier-free guidance:  ε_guided = ε_uncond + w · (ε_cond - ε_uncond)
  Higher guidance weight w → more faithful to prompt, less diverse
```

Chapter 6: Statistics & Probability

Statistics underpins everything in data science — from hypothesis testing in A/B experiments to Bayesian priors in probabilistic models. These topics come up in nearly every interview.

6.1 Probability Basics

```
Bayes' Theorem (THE most important formula in ML):

  P(A|B) = P(B|A) · P(A) / P(B)

  In ML terms:  P(hypothesis | data) ∝ P(data | hypothesis) · P(hypothesis)
                 posterior                 likelihood                prior

Conditional independence:
  X ⊥ Y | Z  means  P(X,Y|Z) = P(X|Z) · P(Y|Z)
  Example: Spam classifier assumes P(word■,word■|spam) = P(w■|spam)·P(w■|spam)

Expectation and Variance:
  E[X] = Σ x·P(x)                    (weighted average of outcomes)
  Var[X] = E[X²] - (E[X])²           (spread around the mean)
  Cov[X,Y] = E[XY] - E[X]·E[Y]      (how X and Y move together)
```

Key distributions to know cold:

Distribution	When it arises	Mean	Variance
Bernoulli(p)	Single coin flip, click/no-click	p	p(1-p)
Binomial(n,p)	Number of successes in n trials	np	np(1-p)
Poisson(λ)	Rare events (support tickets/day)	λ	λ
Normal(μ,σ²)	Heights, measurement errors; CLT limit	μ	σ²
Exponential(λ)	Time between events (service times)	1/λ	1/λ²
Beta(α,β)	Prior over a probability (CTR estimation)	α/(α+β)	αβ/((α+β)²(α+β+1))

6.2 MLE vs MAP — Two Ways to Fit a Model

```
MLE (Maximum Likelihood Estimation):
  θ* = argmax P(data | θ)     No prior — just fit the data

  Example: Gaussian MLE → sample mean and biased variance

MAP (Maximum A Posteriori):
  θ* = argmax P(data | θ) · P(θ)   = MLE + prior

Connection to regularization:
  Gaussian prior P(w) ∝ exp(-λ||w||²)  ↔  L2 (Ridge) regularization
  Laplace prior  P(w) ∝ exp(-λ|w|)    ↔  L1 (Lasso) regularization

Full Bayesian:
  Instead of point estimate, compute full posterior P(θ|data)
  Use MCMC or Variational Inference — expensive but quantifies uncertainty
```

6.3 Hypothesis Testing — The Framework

```
H■ (null hypothesis):         No effect, no difference
H■ (alternative hypothesis):  Effect exists

p-value = P(observing data at least this extreme | H■ true)
  → Small p-value = data is surprising under H■ → reject H■
  → p-value is NOT P(H■ is true)! (common misconception)

Type I error (α = significance level):  Reject H■ when H■ true  (False Positive)
Type II error (β):                      Fail to reject when H■ true (False Negative)
Power = 1 - β = P(detect a true effect)

Typical settings: α = 0.05,  power = 0.80
```

Test	When to use	Assumes normality?
One-sample t-test	Is the mean equal to a reference value?	Yes (or large n)
Two-sample t-test	Are two group means different? (A/B test)	Yes (or large n)
Paired t-test	Before/after for the same units	Yes
Mann-Whitney U	Two groups, non-normal / ordinal data	No (non-parametric)
Chi-squared	Are categorical distributions different?	No
Welch's t-test	Two groups with unequal variances	Approximately

Chapter 7: A/B Testing & Causal Inference

Amazon and Microsoft run thousands of A/B tests simultaneously. This is a topic where deep knowledge directly translates to business impact — and interviewers love it.

7.1 A/B Testing from Start to Finish

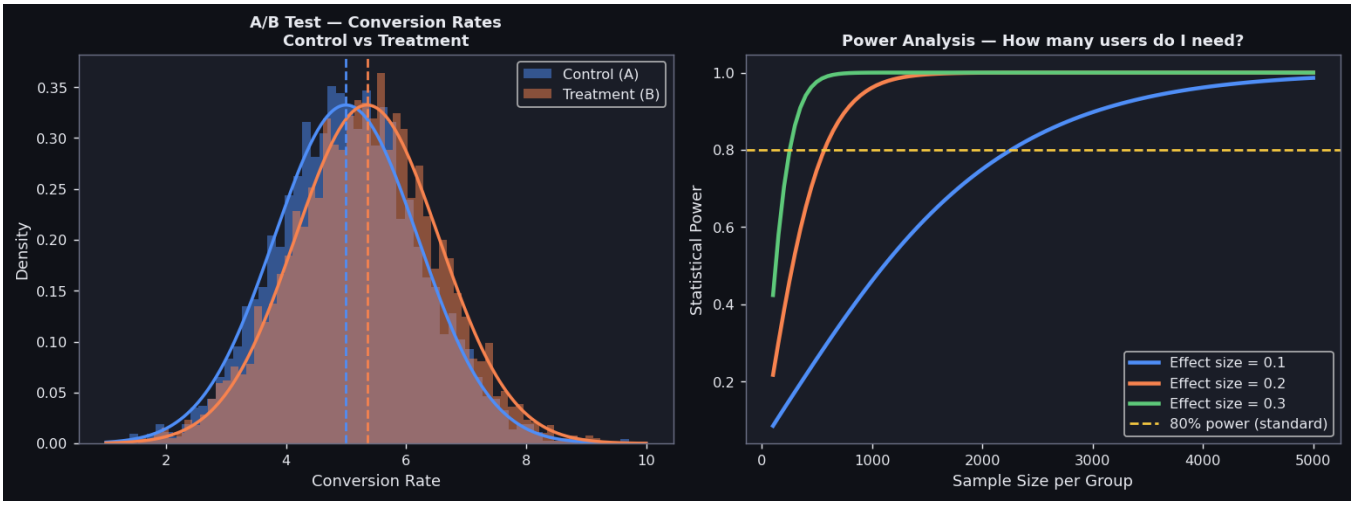

Figure 7.1 — Left: Overlapping distributions of control vs treatment (treatment shows a small +0.35 lift). Right: Power analysis — small effects need many more users.

Step-by-step guide to running an A/B test:

Step	What to do	Common pitfall to avoid
1. Define	State hypothesis + minimum detectable effect (MDE)	Changing hypothesis mid-test ("p-hacking")
2. Sample size	$n = 2\sigma^2(z_\alpha + z_\beta)^2 / \delta^2$ per group	Using too small $\delta \rightarrow$ need unrealistic n
3. Randomize	Assign at user level (not page/session)	Session-level $\rightarrow$ same user sees both variants
4. Run	Run until target n reached (pre-commit)	Peeking at results and stopping early
5. Check SRM	Verify control:treatment ratio is as expected	SRM = data pipeline bug, invalidates test
6. Analyze	Two-sample t-test or z-test on metric	Forgetting multiple testing correction
7. Decide	Report effect size + CI, not just p-value	Significant p but tiny effect (n was huge)

```
Sample size formula:

  n = 2 · σ² · (z_α/2 + z_β)² / δ²

  σ²  = variance of metric (estimate from historical data)
  δ   = minimum detectable effect (what lift do you care about?)
  z_α/2 = 1.96  for α=0.05 (two-sided)
  z_β  = 0.84  for 80% power

CUPED (Variance Reduction — Microsoft):
  Y_cuped = Y - θ · (X_pre - mean(X_pre))
  θ* = Cov(Y, X_pre) / Var(X_pre)
  Variance reduction = (1 - ρ²) × 100%
  Typical reduction: 30-70% → equivalent to having 50-230% more users!
```

7.2 Causal Inference — Beyond A/B Tests

When you cannot randomize (ethical reasons, latency constraints, two-sided markets), you need causal inference methods to estimate treatment effects from observational data.

Method	Idea	Key assumption	Example
RCT / A/B Test	Random assignment	Random assignment (guaranteed)	Button color test
Diff-in-Diff (DiD)	(Treated_after - Treated_before) - (Control change)	Parallel trends	Policy rollout to subset of cities
Propensity Score	Match treated/control by P(T=1|X)	No unmeasured confounders	Compare matched users who did/did not se
Instrumental Variables	Use instrument Z that affects T but not Y directly	Exclusion restriction	Use randomized email invitation as IV for ap
Regression Discontinuity	Compare units just above/below threshold	Continuity of potential outcomes	Age-threshold for feature eligibility

■■ Correlation ≠ Causation example: Ice cream sales and drowning deaths are correlated. They share a confounder: hot weather. Never conclude A causes B without a causal argument.

Chapter 8: Recommendation Systems & NLP

8.1 Recommendation Systems

Amazon's entire product feed, Netflix recommendations, Spotify Discover Weekly — all recommendation systems. They are one of the highest-value ML systems at these companies.

Type	Data used	How it works	Limitation
Collaborative Filtering	User-item interaction matrix	Similar users liked similar items	Cold start (new users/items)
Content-Based	Item features (genre, author, speed)	Recommend items similar to what user liked	Filter bubble
Matrix Factorization	Interaction matrix	Decompose R ≈ PQ■ (user/item embeddings)	Still collaborative
Two-Tower (Deep)	All features	Separate user/item encoders → dot product	Expensive to train
Hybrid	Everything	Blend collaborative + content-based	More complex

```
Matrix Factorization (SVD / ALS):

  R ≈ P · Q■    where P ∈ ■^(users × k), Q ∈ ■^(items × k)

  Loss: min_{P,Q} Σ_{(u,i) observed} (r_ui - p_u · q_i)² + λ(||P||² + ||Q||²)

  SGD update:  p_u ← p_u + η(e_ui · q_i - λ · p_u)  where e_ui = r_ui - p_u·q_i

Two-Tower Neural Architecture (used at Google, Amazon):
  User Tower: user_id embedding + history + demographics → user embedding h_u
  Item Tower: item_id embedding + features + content     → item embedding h_i
  Score: sim(h_u, h_i) = h_u · h_i  or  cosine_similarity(h_u, h_i)
  Retrieval: use FAISS ANN search to find top-K items for a user in real time
```

8.2 Natural Language Processing (NLP)

Topic	Key ideas	Modern approach
Tokenization	Split text into tokens for model input	BPE (GPT) or WordPiece (BERT) — subword tokenization
Word embeddings	Dense vectors for words	Word2Vec, GloVe, FastText (static); BERT (contextual)
Text classification	Assign category to text (spam, sentiment)	Fine-tune BERT / DistilBERT on labeled data
NER	Identify named entities (person, org, location)	Token classification head on BERT
Machine Translation	Translate between languages	Encoder-Decoder transformer (T5, mBART)
Summarization	Generate short summary of long text	BART, T5, GPT with instruction fine-tuning
RAG	Ground LLM answers in retrieved documents	Embed query → retrieve chunks → LLM answers with context

```
BPE (Byte-Pair Encoding) — how GPT tokenizes text:

  Start with character-level vocabulary
  Repeatedly merge the most frequent adjacent pair
  Until vocabulary size V is reached (e.g., V=50,257 for GPT-2)

  "playing" → "play" + "ing"  (common subwords shared across words)
  Handles OOV (out-of-vocabulary) by decomposing into known subwords
```

Chapter 9: ML System Design

ML design rounds are often the deciding factor for senior Applied Scientist roles. They test whether you can translate business problems into ML solutions at production scale.

9.1 The 6-Step ML Design Framework

Step	Questions to ask	Output
1. Problem Formulation	What ML task is this? Regression/Classification/Ranking/Generation? What is the prediction target?	ML objective + success metric
2. Data Strategy	What data exists? How to collect labels? Volume, freshness, bias?	Data pipeline + labeling plan
3. Features	User/item/context features? Real-time vs batch? Training-serving skew?	Feature store design
4. Modeling	Start simple (LR baseline). What complexity is justified? Model architecture + training plan	Model + training plan
5. Evaluation	Offline metrics. Online A/B test. Guardrail metrics. Statistical significance?	Evaluation framework
6. Serving & Monitoring	Latency budget? Batch vs real-time? Drift detection? Retraining?	Serving architecture + alerts

9.2 Example: Design Amazon Search Ranking

Dimension	Decision	Rationale
Problem type	Learning-to-Rank (LTR): listwise NDCG optimization	Position matters in search results
Data	Query logs + human relevance judgments (4-point scale)	Implicit feedback (clicks) + explicit labels
Features	Query-product text match (BM25, BERT-based); product quality	Multi-signal ranking (reviews, sales), user context, real-time
Model	LightGBM + neural re-ranker; start with LambdaMART	Tree models fast, interpretable; neural for top-K re-rank
Metrics (offline)	NDCG@10, MRR for A/B baseline comparison	Position-weighted relevance
Metrics (online)	CTR by rank position, purchase rate, reformulation rate	Real user impact
Serving	<100ms total; feature store for pre-computed features; LightGBM model server	Low latency, high throughput

9.3 MLOps — Keeping Models Healthy in Production

Problem	What it means	How to detect	Fix
Data drift	Feature distribution P(X) shifts over time	Monitor KS-test, PSI score per feature	Retrain on recent data
Concept drift	Relationship P(Y|X) changes	Monitor model performance metrics	Retrain or use online learning
Training-serving skew	Features computed differently in prod vs training	Compare feature stats at train and serve time	Feature store ensures consistency
Model degradation	Gradual performance decay	Daily evaluation on held-out set	Scheduled retraining trigger

```python
# PSI (Population Stability Index) — detect feature drift
def psi(baseline, current, bins=10):
    """PSI < 0.1: no drift. 0.1-0.25: monitor. >0.25: retrain."""
    # Create bins from baseline
    bp = np.percentile(baseline, np.linspace(0, 100, bins+1))
    bp[0], bp[-1] = -np.inf, np.inf

    b_freq = np.histogram(baseline, bins=bp)[0] / len(baseline)
    c_freq = np.histogram(current,  bins=bp)[0] / len(current)

    # Clip to avoid log(0)
    b_freq = np.clip(b_freq, 1e-5, None)
    c_freq = np.clip(c_freq, 1e-5, None)

    return np.sum((c_freq - b_freq) * np.log(c_freq / b_freq))
```

Chapter 10: Coding Patterns for ML Interviews

Applied Scientist coding rounds test numpy/pandas fluency, SQL window functions, and implementing ML concepts from scratch.

10.1 NumPy & Linear Algebra Patterns

```
# Vectorized operations — NEVER loop over numpy arrays
import numpy as np

# Pairwise distances (n x m matrix)
def pairwise_euclidean(A, B):
    # A: (n, d), B: (m, d) -> return (n, m)
    diffs = A[:, None, :] - B[None, :, :]  # broadcast: (n, m, d)
    return np.sqrt((diffs**2).sum(axis=-1))

# Softmax (numerically stable)
def softmax(x):
    x = x - x.max(axis=-1, keepdims=True)  # subtract max for stability
    e = np.exp(x)
    return e / e.sum(axis=-1, keepdims=True)

# Batch matrix multiply with einsum
out = np.einsum("bhd,bde->bhe", Q, K.T)  # (batch, heads, seq, seq)

# Cosine similarity
def cosine_sim(A, B):
    A_norm = A / np.linalg.norm(A, axis=-1, keepdims=True)
    B_norm = B / np.linalg.norm(B, axis=-1, keepdims=True)
    return A_norm @ B_norm.T
```

10.2 SQL — Window Functions (Most Common in Interviews)

```sql
-- Retention analysis: what % of users return after 30 days?
WITH first_use AS (
    SELECT user_id, MIN(event_date) AS cohort_date
    FROM events GROUP BY user_id
),
cohort_activity AS (
    SELECT f.user_id,
           DATE_DIFF(e.event_date, f.cohort_date, DAY) AS day_offset
    FROM first_use f
    JOIN events e USING(user_id)
)
SELECT day_offset, COUNT(DISTINCT user_id) AS retained_users
FROM cohort_activity
GROUP BY 1 ORDER BY 1;

-- Rolling 7-day average revenue per user
SELECT user_id, event_date, revenue,
    AVG(revenue) OVER (
        PARTITION BY user_id
        ORDER BY event_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS rolling_7d_avg,
    RANK() OVER (PARTITION BY country ORDER BY revenue DESC) AS country_rank
FROM sales;
```

10.3 Implement Core Algorithms from Scratch

```python
# K-Means from scratch (vectorized)
def kmeans(X, k, max_iter=100):
    # K-Means++ initialization (better than random)
    centroids = [X[np.random.randint(len(X))]]
    for _ in range(k-1):
        dists = np.min([np.linalg.norm(X - c, axis=1)**2 for c in centroids], axis=0)
        centroids.append(X[np.random.choice(len(X), p=dists/dists.sum())])
    centroids = np.array(centroids)

    for _ in range(max_iter):
        # Assign (E-step)
        dists = np.linalg.norm(X[:, None] - centroids[None, :], axis=-1)
        labels = dists.argmin(axis=1)
        # Update (M-step)
        new_c = np.array([X[labels==j].mean(0) for j in range(k)])
        if np.allclose(new_c, centroids): break
        centroids = new_c
    return labels, centroids

# Scaled dot-product attention (numpy)
def attention(Q, K, V, mask=None):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    if mask is not None: scores[mask] = -1e9
    weights = softmax(scores)
    return weights @ V, weights
```

Chapter 11: Top 100 Interview Questions & Answers

These are the most frequently asked questions at Amazon and Microsoft Applied Scientist interviews, organized by topic. Each answer is written in plain language with key formulas.

ML THEORY & FUNDAMENTALS

Q1. What is the bias-variance tradeoff?

Every model's error = Bias² + Variance + Irreducible noise. Bias is systematic error from wrong assumptions (underfit). Variance is sensitivity to training data (overfit). Fix high bias: more complex model. Fix high variance: more data, regularization, ensemble. Diagnose via learning curves: both train/val high = bias. Gap between them = variance.

Q2. What is overfitting and how do you fix it?

Overfitting: model memorizes training data, fails on new data. Signs: train accuracy >> val accuracy. Fixes: (1) More training data. (2) Regularization (L1/L2, dropout). (3) Simpler model. (4) Early stopping. (5) Data augmentation. (6) Ensemble methods. (7) Cross-validation during tuning.

Q3. Explain L1 vs L2 regularization. When would you use each?

L2 (Ridge): adds $\lambda\Sigma w_i^2$ — shrinks weights but never to exactly zero. Good when all features relevant. L1 (Lasso): adds $\lambda\Sigma|w_i|$ — some weights become exactly zero (feature selection). Good when many features are irrelevant. Geometric reason: L1 ball has corners on axes — optimal solution tends to land on a corner (zero weight). Elastic Net combines both — good when features are correlated.

Q4. What is gradient descent? What are the different variants?

Iteratively update weights in the direction of negative gradient: $w \leftarrow w - \eta \cdot \nabla L$. Batch GD: uses all data (slow but stable). SGD: uses 1 sample (noisy, can escape local minima). Mini-batch: uses 32-512 samples — standard in deep learning. Adam adapts lr per weight using running mean/variance of gradients — most popular optimizer.

Q5. What is cross-validation? When do you use stratified k-fold?

k-Fold: split data into k folds, train on k-1, test on remaining 1, repeat. Average results. Gives honest estimate of generalization. Stratified k-fold: maintains class proportions per fold — use with imbalanced datasets (e.g., fraud detection). Time series: never shuffle — use expanding window or forward-chaining split.

Q6. Explain the curse of dimensionality.

As feature dimensions grow, data becomes exponentially sparse. Every point is far from every other. Distance metrics lose discriminative power. Density estimation fails. Need exponentially more data to maintain coverage. Solutions: feature selection, PCA/dimensionality reduction, L1 regularization (sparsity), domain knowledge to select relevant features only.

Q7. What is data leakage and how do you detect it?

Leakage: future or target information leaks into training features. Types: (1) Target leakage: feature is computed using the target (e.g., "account_closed_flag" to predict churn). (2) Temporal leakage: using future data to predict the past. (3) Train-test leakage: fitting scaler on full dataset. Detection: suspiciously high CV score, features with correlation > 0.95 with target. Fix: use sklearn Pipeline, time-aware splits, careful feature engineering review.

Q8. What is the EM algorithm? Where is it used?

Expectation-Maximization: iterative algorithm for models with latent (hidden) variables. E-step: compute expected complete-data log-likelihood given current parameters. M-step: maximize this expectation to update parameters. Monotonically increases likelihood. Converges to local optimum. Applications: GMM clustering, HMM training, missing data imputation, topic modeling (LDA).

Q9. How do you handle class imbalance?

Data-level: SMOTE (synthetic samples by interpolating minority class), random oversampling, undersampling, Tomek links. Algorithm-level: class_weight="balanced" in sklearn, threshold adjustment. Loss-level: focal loss (down-weights easy examples). Evaluation: use AUC-PR (not ROC) — ROC is overly optimistic for extreme imbalance.

Q10. Explain information gain and Gini impurity.

Both measure how good a split is in a decision tree. Gini: $G = 1 - \Sigma p_{\blacksquare}^2$ — probability of misclassifying a random sample. Entropy / IG: $H = -\Sigma p_{\blacksquare} \log_{\blacksquare}(p_{\blacksquare})$. Information Gain = H(parent) - weighted H(children). Both give similar splits in practice. Gini is faster to compute (no log). sklearn uses Gini by default.

Q11. What is the difference between parametric and non-parametric models?

Parametric: fixed number of parameters regardless of data size (logistic regression, Gaussian). Non-parametric: complexity grows with data (k-NN, kernel SVM, decision trees without max_depth). Parametric: fast, interpretable, needs assumptions. Non-parametric: flexible, fewer assumptions, slower for large data.

Q12. What is model calibration? How do you calibrate a model?

A calibrated model's probability outputs match actual frequencies. If it says P=0.8, then 80% of such instances should be positive. Measure: reliability diagram (plot mean predicted prob vs observed fraction per bin). Fix: Platt scaling (fit logistic regression on outputs), isotonic regression, or temperature scaling (divide logits by scalar T, tuned on validation set).

MACHINE LEARNING ALGORITHMS

Q13. Explain SVMs. Why does the kernel trick work?

SVM finds the maximum-margin hyperplane between classes. Only "support vectors" (nearest points) determine the boundary. Kernel trick: $K(x,z) = \phi(x)_{\blacksquare}\phi(z)$ computes inner products in high-dimensional feature space without explicitly computing the mapping ϕ. RBF kernel $K(x,z)=\exp(-\gamma||x-z||^2)$ corresponds to infinite-dimensional feature space. Mercer's theorem: K is valid iff it is positive semi-definite.

Q14. How does Random Forest work? Why does it reduce variance?

Trains many decision trees in parallel, each on a random bootstrap sample with random feature subsets at each split (typically $\sqrt{d}$ features). Prediction: majority vote. Variance reduction: trees are decorrelated (different data + different features), and averaging uncorrelated predictions reduces variance. OOB error: ~37% of samples unused per tree $\rightarrow$ free validation estimate.

Q15. How does Gradient Boosting differ from Random Forest?

RF: parallel trees, reduces variance via averaging. GB: sequential trees, each fits the negative gradient (residuals) of the previous ensemble. GB reduces bias; RF reduces variance. GB usually higher accuracy but requires careful tuning (learning rate, depth, early stopping). RF is more robust out-of-the-box.

Q16. Explain XGBoost's key innovations over standard gradient boosting.

(1) Second-order Taylor expansion for better gradient estimates. (2) Regularized objective: penalizes tree complexity (number of leaves, leaf weight magnitude). (3) Column and row subsampling. (4) Parallel split-finding by pre-sorting columns. (5) Built-in missing value handling (learns optimal direction). (6) Early stopping on validation metric. (7) Shrinkage (learning rate).

Q17. When would you choose k-NN over a neural network?

k-NN: very small datasets where deep learning overfits; when you need immediate updates (no retraining — just add data points); when interpretability is critical ("which past examples are closest to this new one?"). Neural net: large datasets, raw inputs (images, text), when deep features help, when inference speed matters (k-NN is $O(nd)$ per query). In practice: almost always use tree models or neural nets over k-NN in production.

Q18. Explain PCA. What are its limitations?

PCA projects data onto directions of maximum variance via eigendecomposition of covariance matrix. Limitations: (1) Only captures LINEAR structure. (2) Variance $\neq$ importance — the highest-variance direction may not be the most predictive. (3) Sensitive to outliers. (4) Loses interpretability. (5) Need to standardize features first (or high-scale features dominate). Alternatives: kernel PCA (non-linear), autoencoders (deep non-linear).

Q19. Explain K-Means clustering. What is K-Means++?

K-Means minimizes within-cluster sum of squares. E-step: assign points to nearest centroid. M-step: update centroids to cluster means. Repeats until convergence. K-Means++: smarter initialization — choose centroids with probability $\propto D(x)^2$ (distance to nearest existing centroid). Result: faster convergence, better solutions. Choosing k: elbow method (plot WCSS vs k), silhouette score.

Q20. What is the difference between GMM and K-Means?

K-Means: hard assignments (each point belongs to exactly one cluster), assumes spherical clusters, minimizes within-cluster variance. GMM: soft assignments (each point has probability of belonging to each cluster), fits elliptical Gaussians, parameters estimated by EM. GMM is more expressive but slower and has more parameters. GMM is equivalent to K-Means in the limit of small covariances.

Q21. Explain DBSCAN. When is it better than K-Means?

DBSCAN: density-based clustering. Core point: has $\geq$ MinPts neighbors within radius ε. Border: within ε of a core point. Noise: neither. Advantages over K-Means: (1) Does not need k specified. (2) Finds arbitrary-shaped clusters. (3) Identifies outliers as noise. Disadvantage: fails with varying densities (use HDBSCAN), sensitive to ε and MinPts.

Q22. What is the purpose of dimensionality reduction before modeling?

(1) Speed: fewer features → faster training and prediction. (2) Reduce overfitting: curse of dimensionality makes distance metrics unreliable. (3) Remove noise: low-variance components often contain noise. (4) Visualization: project to 2D/3D for exploration. (5) Feature store efficiency: compress embeddings for similarity search. PCA: linear, fast. t-SNE/UMAP: visualization only. Autoencoders: non-linear.

DEEP LEARNING

Q23. Explain backpropagation. How does it work?

Backprop applies the chain rule to compute gradients of the loss w.r.t. all weights. Forward pass: compute activations layer by layer. Backward pass: compute $\partial L/\partial W$ at each layer by propagating error backwards. $\partial L/\partial W_l = \partial L/\partial A_L \cdot \partial A_L/\partial Z_L \cdot \ldots \cdot \partial Z_l/\partial W_l$ (chain rule). Key insight: reuse intermediate computations — efficient $O(n)$ vs $O(n^2)$ naive.

Q24. What is the vanishing gradient problem? How is it solved?

Deep networks with sigmoid/tanh: gradients shrink as they flow backwards through layers. Sigmoid derivative ≤ 0.25 → after 10 layers, gradient $\leq (0.25)^{10} \approx 10^{-6}$. Weights in early layers barely update — network does not learn. Solutions: (1) ReLU: derivative = 1 for $x>0$. (2) Residual connections: +1 term in gradient. (3) Batch normalization: keeps activations in healthy range. (4) He/Xavier initialization: keeps initial gradient magnitudes appropriate.

Q25. What is batch normalization? Why use layer norm for transformers?

BatchNorm: normalize each feature across the batch dimension (zero mean, unit variance), then scale with learnable γ, β. Benefits: allows higher learning rates, slight regularization. Problem for transformers: sequence length varies; batch size may be 1 at inference. LayerNorm: normalize across feature dimension per token — independent of batch size. PreLayerNorm (normalize before sublayer) is more stable than PostLayerNorm.

Q26. Explain the transformer's self-attention mechanism.

Each token creates Query (Q), Key (K), Value (V) vectors via linear projections. Score: how well each query matches each key: scores = QK■/√d_k. Attention weights: softmax(scores). Output: weighted sum of values. Multi-head: run h attention operations in parallel, concatenate results. Each head can capture different relationships (syntax, semantics, coreference). Complexity: $O(n^2d)$ — quadratic in sequence length.

Q27. What is the difference between BERT and GPT?

BERT: encoder-only, bidirectional (sees full context both ways). Pre-trained with masked LM (predict 15% random tokens). Good for understanding tasks: classification, NER, extractive QA. GPT: decoder-only, autoregressive (left-to-right only). Pre-trained with next token prediction. Good for generation tasks: chat, code, completion. Modern trend: decoder-only (GPT-style) dominates — emergent few-shot and instruction-following abilities.

Q28. What is LoRA? Why is it efficient for fine-tuning?

LoRA: freeze pre-trained weights W■, add trainable low-rank factorization $\Delta W=BA$. $B \in$ ■^(d×r), $A \in$ ■^(r×k), $r << d$. Init: B=0 (starts as identity). At inference: merge W = W■ + BA (zero overhead). Typical: r=8 on 4096×4096 matrix = 65K params vs 16.7M → 256× fewer. QLoRA additionally quantizes W■ to 4-bit NF4

format, reducing VRAM by ~4×.

Q29. How does RLHF work to align LLMs?

Step 1: SFT — fine-tune on human-written demonstrations to get a base instruction follower. Step 2: Reward Model — collect pairs of model responses with human preference labels, train RM to predict which response is better (Bradley-Terry model). Step 3: PPO — maximize RM score while staying close to SFT model via KL divergence penalty. DPO (alternative): directly optimize preference data as supervised learning — simpler, often as good.

Q30. Explain diffusion models. How do they generate images?

Forward process: gradually add Gaussian noise to clean image over T steps until pure noise. Reverse process: train a U-Net to predict and remove the noise at each step. Training objective: predict the noise ε that was added: $L = ||\varepsilon - \varepsilon_\theta(x_t, t)||^2$. Generation: start from pure noise, run reverse process T times. DDIM: deterministic sampling in ~20-50 steps. Classifier-free guidance: blend conditional and unconditional noise predictions for quality-diversity tradeoff.

Q31. What is dropout? How does inverted dropout work?

Dropout: during training, randomly zero each neuron with probability p. Forces the network to learn redundant representations (no co-adaptation). Inverted dropout: divide active neurons by (1-p) during training so the expected output is unchanged. At test time, use all neurons without any scaling needed. Typical values: p=0.5 for dense layers, p=0.1 for transformers. Dropout is a form of ensemble averaging over exponentially many sub-networks.

Q32. Explain residual connections. Why do they help?

ResNet: $H(x) = F(x) + x$. Skip connection adds identity to the transformation. Gradient flow: $\partial L/\partial x = \partial L/\partial H \cdot (\partial F/\partial x + 1)$. The +1 term ensures gradient is at least 1 everywhere — no vanishing. Interpretation: network learns residuals (small corrections) rather than full transformations. F(x) starts near 0 (He init) → layers start as near-identity, easy to train deep stacks. Enables training 1000+ layer networks.

STATISTICS & PROBABILITY

Q33. What is Bayes' theorem? Give an ML example.

$P(A|B) = P(B|A)\cdot P(A) / P(B)$. In ML: $P(\theta|\text{data}) \propto P(\text{data}|\theta)\cdot P(\theta)$. Example — disease test: P(disease) = 0.001 (base rate), P(positive|disease) = 0.99. If test is positive: P(disease|+) = (0.99×0.001) / (0.99×0.001 + 0.01×0.999) ≈ 9%. Counterintuitive because base rate is low! Always check the prior.

Q34. What is the Central Limit Theorem? Why does it matter?

CLT: sum/mean of n iid random variables → Normal distribution as $n \to \infty$, regardless of the underlying distribution (requires finite mean and variance). Why it matters: (1) Foundation of t-tests and z-tests. (2) Sampling distributions of estimators are approximately normal. (3) Justifies using normal-based CI for A/B tests even with non-normal metrics. Sample means are well-behaved for $n \geq 30$ (rule of thumb).

Q35. Explain Type I and Type II errors. What is the tradeoff?

Type I (α = false positive rate): reject H■ when it is true → ship a bad feature. Type II (β = false negative rate): fail to reject when H■ is true → miss a good feature. Power = 1 - β = probability of detecting a true

effect. Tradeoff: lowering α (fewer false positives) increases β (more false negatives). Business context: for minor features, higher α is acceptable. For safety-critical or expensive changes, use lower α.

Q36. What is a p-value? What is a common misconception?

p-value = P(observing data this extreme or more extreme | H■ is true). Common misconceptions: (1) p is NOT P(H■ is true) — it tells you nothing about H■ probability. (2) $p > 0.05$ does not mean "H■ is true" — only "insufficient evidence to reject." (3) Small p $\neq$ large effect — with huge n, trivially small effects become significant. Always report effect size and confidence interval alongside p-value.

Q37. How do you design an A/B test? What pitfalls exist?

Steps: define hypothesis + MDE, compute sample size, randomize at user level, run until n reached, check SRM, run t-test, report CI. Pitfalls: (1) Peeking: stopping early when results look good inflates Type I error. (2) Multiple testing: testing 20 metrics $\rightarrow$ expect 1 false positive at α=0.05. (3) Network effects: users affect each other (SUTVA violation). (4) Novelty effect: new feature gets artificial early boost. (5) SRM: unequal traffic split = data pipeline bug.

Q38. What is CUPED? Why is it useful?

CUPED (Microsoft): use pre-experiment data to reduce metric variance. Y_cuped = Y - θ(X_pre - E[X_pre]) where θ = Cov(Y, X_pre) / Var(X_pre). Y_cuped has same expectation as Y but lower variance. Reduction: $(1 - \rho^2)$ where ρ = correlation between Y and X_pre. If ρ=0.7, variance reduced by 51% $\rightarrow$ equivalent to doubling sample size. Free statistical power from existing data!

Q39. Explain difference-in-differences (DiD).

Causal method for observational studies. ATT = (Treated_after - Treated_before) - (Control_after - Control_before). Differences out time trends (both groups have same time trend) and time-invariant confounders. Key assumption: parallel trends — in absence of treatment, both groups would have moved in parallel. Check with pre-period event study plot. Use case: policy rolled out to some regions, evaluate impact vs regions without.

Q40. What is MLE? What is its connection to cross-entropy loss?

MLE finds parameters θ maximizing P(data | θ) = Π P(x■ | θ). Equivalently, maximize log-likelihood: Σ log P(x■ | θ). Connection to cross-entropy: For classification with softmax output, minimizing cross-entropy L = -Σ y■ log ■■ is exactly MLE under categorical distribution. L2 regularization = Gaussian prior on weights (MAP). L1 regularization = Laplace prior on weights.

SYSTEM DESIGN & MLOPS

Q41. Design a fraud detection system.

Multi-layer: (1) Rule-based (<1ms): known bad IPs, velocity checks. (2) Real-time ML (<100ms): LightGBM on transaction + user features (amount, location change, merchant category, velocity past 1h/24h). (3) Batch deep learning: graph-based, sequence models. Imbalance: 0.1% fraud — use class weights, AUC-PR metric. Labels: delayed chargebacks $\rightarrow$ need to handle temporal mismatch. Monitoring: daily offline eval on held-out set, champion-challenger deployment.

Q42. Design a recommendation system for Amazon.

Two-stage pipeline: (1) Candidate generation: two-tower model (user + item embeddings), FAISS ANN search retrieves top-500 candidates. Also: collaborative filtering, session-based, trending. (2) Ranking: gradient boosted model with 100s of features. Features: user history (purchases, views, search), item attributes, context (device, time). Optimize: long-term value not just CTR. Cold start: content-based fallback for new items, popularity for new users. Metrics: CTR, add-to-cart rate, revenue, long-term engagement.

Q43. How do you handle training-serving skew?

Root cause: features computed differently at training time (batch, historical) vs serving time (real-time, online). Prevention: (1) Feature store: same feature computation logic for both. (2) Shadow validation: compare feature distributions between training and prod logs. (3) Same code path: use same preprocessing functions, not separate scripts. Detection: monitor feature distribution shifts in production (PSI, KS test). Fix: log production features, periodically retrain on production distribution.

Q44. How do you detect and handle model drift in production?

Data drift: P(X) changes — monitor feature distributions (PSI < 0.1 ok, >0.25 retrain). Concept drift: P(Y|X) changes — monitor performance metrics over time. Label drift: P(Y) changes — monitor prediction distribution. Detection: automated dashboards, alerting thresholds. Response: (1) Triggered retraining when PSI > threshold. (2) Scheduled retraining (weekly). (3) Online learning for fast-changing patterns.

Q45. What is a feature store and why is it important?

Centralized repository for ML features with: (1) Offline store (data warehouse) for batch training with point-in-time correctness. (2) Online store (Redis/DynamoDB) for low-latency (<10ms) serving. (3) Feature registry: metadata, ownership, lineage. Why important: (1) Prevents training-serving skew. (2) Feature reuse across teams (avoid duplicate computation). (3) Point-in-time correctness prevents data leakage in training. Examples: Feast (open source), Tecton, AWS SageMaker Feature Store.

Q46. Explain model deployment strategies.

Shadow: new model runs in parallel without serving traffic — validate before exposure. Canary: route 1-5% traffic to new model — catch bugs with minimal risk. Blue-Green: full instant traffic switch with rollback capability. Multi-armed bandit: dynamically allocate traffic to best-performing variant. A/B test rollout: controlled comparison with statistical significance test. Gradual ramp: 1% → 5% → 20% → 100% with monitoring at each stage.

Q47. How would you evaluate fairness in an ML model?

Fairness metrics: (1) Demographic parity: equal positive rate across groups. (2) Equalized odds: equal TPR and FPR across groups. (3) Calibration: predicted probabilities match actual rates per group. Detection: slice-based evaluation by demographic attributes. Mitigation: pre-processing (reweighting), in-processing (fairness constraints), post-processing (threshold calibration per group). Key insight: different fairness definitions are often mathematically incompatible.

Q48. What metrics would you use to evaluate a search ranking system?

NDCG@k: DCG = Σ (2^rel_i - 1)/log■(i+1), normalized by ideal DCG. Accounts for position — top results matter more. MRR: mean 1/rank of first relevant result — good for navigational queries. MAP: mean average precision across queries. Online: CTR by position (click-through rate), dwell time, reformulation rate (user searches again = failure), task completion. Gold standard: human relevance judgments on random query samples.

CODING & IMPLEMENTATION

Q49. Implement logistic regression from scratch.

```
def sigmoid(z): return 1 / (1 + np.exp(-z))
def fit(X, y, lr=0.01, n_iter=1000):
    w, b = np.zeros(X.shape[1]), 0
    for _ in range(n_iter):
        yhat = sigmoid(X @ w + b)
        w -= lr * X.T @ (yhat - y) / len(y)
        b -= lr * (yhat - y).mean()
    return w, b
```

Q50. How would you compute cosine similarity efficiently for 1M embeddings?

Brute force: $O(n^2)$ — infeasible. Use Approximate Nearest Neighbor (ANN): (1) Normalize all embeddings to unit length. (2) Use FAISS (Facebook): IndexFlatIP for exact, IndexIVFPQ for approximate. (3) HNSW graph: O(log n) search with 95%+ recall. faiss.normalize_L2(embeddings); index = faiss.IndexHNSWFlat(d, 32); index.add(embeddings); D, I = index.search(query, k=10).

Q51. Write SQL to compute 30-day retention for user cohorts.

```
WITH first_action AS (SELECT user_id, MIN(date) cohort FROM events GROUP BY 1),
retention AS (SELECT f.user_id, DATE_DIFF(e.date, f.cohort, DAY) AS day
  FROM first_action f JOIN events e USING(user_id))
SELECT cohort, day, COUNT(DISTINCT user_id) AS users
FROM retention GROUP BY 1, 2 ORDER BY 1, 2
```

Q52. How would you implement early stopping?

Track best validation loss. If validation loss does not improve for `patience` steps, stop and restore best weights. Implementation: best_val = inf; patience_counter = 0. Each epoch: if val_loss < best_val - delta: save weights, reset counter. Else: counter += 1; if counter >= patience: stop and reload best weights.

Q53. Implement k-fold cross-validation from scratch.

```
def kfold_cv(X, y, model, k=5):
    n = len(X); idx = np.random.permutation(n)
    folds = np.array_split(idx, k)
    scores = []
    for i in range(k):
        val = folds[i]
        tr = np.concatenate([folds[j] for j in range(k) if j!=i])
        model.fit(X[tr], y[tr])
        scores.append(eval(model.predict(X[val]), y[val]))
    return np.mean(scores), np.std(scores)
```

ADVANCED ML & RESEARCH TOPICS

Q54. What is multi-task learning? When does it help?

Train one model on multiple related tasks simultaneously, sharing representations. Helps when: tasks share underlying structure, one task has limited data, auxiliary tasks provide useful inductive bias. Examples: NLP model trained jointly on NER + POS + parsing; recommendation model sharing user behavior signals across click and purchase prediction. Risk: negative transfer when tasks conflict. Mitigation: task-specific heads, gradient surgery, task weighting.

Q55. Explain transfer learning. When does fine-tuning fail?

Use knowledge from pre-trained model for new task. Approaches: fine-tune all, fine-tune last layers, feature extraction. Fails when: source and target domains are very different (ImageNet → medical X-ray). Source model too small for target. Target labels contradict source representations. Mitigation: gradual unfreezing (fine-tune from output backward). Domain adaptation techniques: adversarial alignment, DANN.

Q56. What is knowledge distillation?

Transfer a large teacher model's "soft" knowledge to a smaller student. Student learns from teacher's softmax probabilities (not just hard labels). Temperature T softens: $p_i = \exp(\text{logit}_i/T)/\Sigma\exp(\text{logit}_j/T)$. Loss: $\alpha \cdot \text{CE}(\text{student, hard_labels}) + (1-\alpha) \cdot \text{KL}(\text{student} \,||\, \text{teacher_soft})$. Benefits: student learns inter-class similarity ("dark knowledge"). DistilBERT: 40% smaller BERT, 97% of performance.

Q57. What is the difference between online and batch learning?

Batch learning: train on entire dataset, deploy static model, periodic retraining. Simple, stable, reproducible. Online learning: update model continuously on each new data point. Handles concept drift automatically. Challenges: catastrophic forgetting, noise sensitivity, harder to debug. Hybrid: mini-batch updates on rolling windows — practical compromise. Used in ad click prediction (needs sub-day updates), fraud detection.

Q58. Explain attention vs self-attention vs cross-attention.

Attention (general): query attends to a set of keys/values. Self-attention: Q, K, V all come from the same sequence — tokens attend to each other. Used in transformer encoders (BERT) and decoders (GPT). Cross-attention: Q comes from one sequence (decoder), K/V from another (encoder output). Used in encoder-decoder models (T5, BART) to allow decoder to attend to encoder context.

Q59. What are mixture of experts (MoE) models?

Replace dense FFN layers with N expert FFNs. A learned router selects top-k experts per token (typically k=1 or 2). Only activated experts run → same compute as dense model with k/N of experts. Benefits: scale parameters without proportionally scaling compute. Mixtral 8x7B: 8 experts, activates 2 per token → effective size of 12B but quality of 47B. Challenges: load balancing (auxiliary loss), communication in distributed training.

Q60. Explain SHAP values and their advantages over feature importance.

SHAP: Shapley Additive exPlanations from game theory. Assigns each feature a contribution to each individual prediction. Properties: efficiency (contributions sum to prediction), symmetry, dummy, additivity. TreeSHAP: exact polynomial-time computation for tree models. Advantages over impurity importance: (1) Consistent — if model increases feature importance, SHAP increases too. (2) Per-instance explanations. (3) Captures interaction effects. (4) Not biased toward high-cardinality features.

Q61. What is semi-supervised learning? When is it useful?

Leverages large unlabeled dataset + small labeled dataset. Methods: (1) Pseudo-labeling: train on labeled, predict unlabeled, retrain on high-confidence predictions. (2) Consistency regularization: model predictions

should be stable under augmentation. (3) Self-supervised pre-training then fine-tune (BERT, SimCLR). Useful when: labeling is expensive (medical images, legal documents), unlabeled data is abundant. Risks: confirmation bias with pseudo-labeling.

Q62. Explain contrastive learning (SimCLR, CLIP).

SimCLR: take two augmented views of the same image, train encoder to bring them close, push representations of different images apart. Loss: InfoNCE. Large batch provides many negatives. CLIP: align image and text pairs. Maximize similarity of matched pairs, minimize for non-matched. Enables zero-shot image classification by comparing image embedding to text descriptions of classes. Key insight: rich semantics emerge from solving hard comparison tasks on diverse data.

Q63. What is catastrophic forgetting? How is it addressed?

When a neural network learns new tasks, it overwrites weights needed for old tasks — "forgets" previously learned information. Solutions: (1) Elastic Weight Consolidation (EWC): penalize changes to important weights. (2) Replay methods: store examples from old tasks, interleave during training. (3) Progressive neural networks: new column per task, freeze old columns. (4) LoRA adapters: task-specific adapters on frozen backbone — no interference.

Q64. What is Retrieval-Augmented Generation (RAG)?

Ground LLM responses in retrieved documents to reduce hallucinations and provide up-to-date/private knowledge. Pipeline: (1) Embed query with dense encoder. (2) Retrieve top-k relevant documents from vector database (FAISS, Pinecone, Weaviate). (3) Prepend retrieved context to prompt. (4) LLM generates grounded answer. Advantages: no retraining needed, answers are verifiable, knowledge is updatable. Challenges: retrieval quality, context window limits, answer synthesis quality.

Q65. How does speculative decoding speed up LLM inference?

Problem: LLM generation is slow (autoregressive — one token at a time). Speculative decoding: (1) Small draft model generates N tokens quickly. (2) Large target model verifies all N tokens in a single forward pass. (3) Accept tokens up to the first disagreement. Speedup: 2-5× depending on acceptance rate and model size ratio. Works because the target model can score N tokens in parallel in one pass, even though it generates them sequentially.

Q66. What is the difference between precision@k and NDCG@k?

Precision@k: fraction of top-k results that are relevant. Binary: either relevant or not. Does not account for position within the top-k. NDCG@k: normalized discounted cumulative gain. Accounts for: (1) Graded relevance (not just binary). (2) Position: earlier results are more valuable (log discount). DCG = Σ■■■■ (2^rel■ - 1) / log■(i+1). NDCG = DCG / IDCG (normalized by ideal). Use NDCG for search/ranking where both position and graded relevance matter.

Q67. Explain the differences between VAEs and GANs.

VAE: encoder-decoder with variational bottleneck. Explicit likelihood maximization (ELBO). Smooth, structured latent space. Blurry outputs. Inference: simple (one encoder forward pass). GAN: generator vs discriminator game. Implicit likelihood — no direct optimization of P(x). Sharp, realistic outputs. No structured latent space. Hard to train (mode collapse, instability). No encoder by default. Diffusion models now largely replace both for image generation.

Q68. What are guardrail metrics in A/B testing?

Metrics you monitor to ensure the experiment does not harm key user experiences, even when optimizing a primary metric. Example: optimize for ad CTR (primary) but monitor: page load time, user session length, unsubscribe rate, click-through to purchase (conversion). Even if the primary metric improves, if a guardrail degrades, do not ship. Guardrails represent the business constraints that must not be violated. Typically use a lower significance threshold for guardrails (α=0.01 not 0.05).

Q69. What is label smoothing? When should you use it?

Replace hard 0/1 labels with soft targets: ■ = $(1-\varepsilon)\cdot y + \varepsilon/K$ (K = num classes). Prevents model from being overconfident (large logit differences). Improves calibration: predicted probabilities become more reliable. Regularization effect: reduces overfitting when training with limited data. Standard: $\varepsilon = 0.1$. Used in image classification (InceptionV4), machine translation. Relationship: equivalent to adding a KL divergence from uniform distribution to the loss.

Q70. How does Word2Vec work? What is skip-gram?

Word2Vec learns word vectors where similar words are close in embedding space. Skip-gram: given a center word, predict surrounding context words in a window. CBOW: given context, predict center word. Negative sampling: instead of full softmax over vocabulary (50K words), predict the target word vs k random noise words (k=5-20). This makes training tractable. Properties: king - man + woman ≈ queen. Limitation: static embeddings — same vector for a word regardless of context (replaced by BERT).

Q71. Explain the attention mechanism in transformers. What is the complexity?

Attention(Q, K, V) = softmax(QK■/$\sqrt{d_k}$) · V. Time complexity: $O(n^2 \cdot d)$ — each of n positions attends to all n positions. Space complexity: $O(n^2)$ — must store n×n attention matrix. This is the bottleneck for long sequences. Efficient alternatives: FlashAttention (IO-aware, O(n) memory with recomputation), Sparse attention (attend to local window), linear attention (kernel approximation O(n)), GQA/MQA (grouped/multi-query attention — fewer K,V heads).

Q72. What is the role of the temperature parameter in LLM generation?

Temperature T scales logits before softmax: p_i = exp(logit_i/T) / Σexp(logit_j/T). T=1: default distribution. T<1: sharpens distribution → more confident, less diverse. T>1: flattens distribution → more diverse, more random. T→0: greedy (always pick argmax). Top-p (nucleus) sampling: sample from the smallest set of tokens whose total probability ≥ p. Top-k: sample from the k most likely tokens. Typical settings: T=0.7 for chat, T=0 for factual tasks, T=1 for creative writing.

Q73. How would you reduce hallucination in LLMs?

RAG: ground model in retrieved factual documents. Fine-tuning: train on domain-specific accurate data. Calibration: ensure model knows what it does not know (output "I don't know"). Chain-of-thought prompting: force reasoning steps before answer. Self-consistency: sample multiple generations, take majority vote. Fact verification: post-hoc check outputs against knowledge base. RLHF: reward factual, penalize confabulation. Smaller context: long contexts degrade factuality.

Q74. What is prompt engineering? What are key techniques?

Design of prompts to elicit desired behavior from LLMs without weight updates. Techniques: (1) Zero-shot: direct instruction. (2) Few-shot: 3-10 examples in prompt. (3) Chain-of-thought (CoT): "think step by step" → forces reasoning. (4) Self-consistency: sample CoT multiple times, majority vote answer. (5) Role prompting: "You are an expert radiologist..." (6) Structured output: "Respond in JSON: {field: value}". (7) ReAct: interleave reasoning and tool use (Thought / Action / Observation).

Q75. Explain quantization in deep learning. What is INT8 and INT4 quantization?

Reduce precision of weights/activations from FP32 → INT8 or INT4. Benefits: 4-8× memory reduction, 2-4× inference speedup. INT8: 8-bit integers. Minimal quality loss. Standard for deployment. INT4: 4-bit. More aggressive. GPTQ, AWQ, NF4 (QLoRA) methods. Post-training quantization (PTQ): calibrate with small dataset, no retraining. Quantization-aware training (QAT): simulate quantization during training, better quality. Key metric: perplexity change or benchmark score degradation.

Q76. What is positional encoding in transformers?

Transformers process all tokens in parallel — no inherent notion of order. Positional encoding adds position information to token embeddings. Sinusoidal (original): PE(pos, 2i) = sin(pos/10000^(2i/d)). Learned: trainable embedding table per position (BERT). RoPE (Rotary): encode position as rotation in embedding space. Used in LLaMA — better for long contexts, can extrapolate beyond training length. ALiBi: add linear bias to attention scores based on distance.

Q77. What is the difference between encoder-only, decoder-only, and encoder-decoder models?

Encoder-only (BERT): bidirectional attention, sees full context. Best for: understanding tasks (classification, NER, semantic similarity). No generation capability. Decoder-only (GPT, LLaMA): causal/autoregressive, left-to-right attention. Best for: generation, instruction following, reasoning. Encoder-decoder (T5, BART): encoder processes input bidirectionally, decoder generates output autoregressively with cross-attention to encoder. Best for: translation, summarization, structured prediction.

Q78. How do you evaluate LLM outputs?

Automated: perplexity (language model quality), BLEU/ROUGE (translation/summarization — imperfect), BERTScore (semantic similarity via BERT embeddings), MMLU (knowledge breadth), HumanEval (coding), TruthfulQA (factuality). Human evaluation: pairwise preference (A vs B), absolute rating (1-5). Gold standard but expensive and slow. LLM-as-judge: use GPT-4 to score outputs on rubric. MT-Bench: diverse multi-turn conversation benchmark. AlpacaEval: compare to reference model on instruction-following.

Q79. What is Flash Attention?

Tiling algorithm for computing attention with O(n) memory (vs standard $O(n^2)$). Key insight: attention computation is memory-bandwidth bound, not compute bound. Standard attention: reads/writes full n×n attention matrix to GPU HBM. FlashAttention: tiles computation, keeps intermediate results in fast SRAM cache. Result: 2-4× faster, O(n) memory — enables longer sequences. FlashAttention-2: further optimized for H100 GPUs. Used in essentially all modern LLM training.

Q80. What is instruction tuning?

Fine-tune a language model on (instruction, response) pairs to follow natural language instructions. FLAN (Google): fine-tune T5 on 60+ NLP tasks phrased as instructions — massive zero-shot improvement. InstructGPT: SFT on human-written demonstrations, then RLHF. Alpaca: fine-tune LLaMA on 52K ChatGPT-generated instruction pairs. Why it works: exposes model to diverse instruction formats, aligns output style with human expectations. Key data factors: diversity of tasks, quality of demonstrations, format consistency.

BEHAVIORAL & LEADERSHIP PRINCIPLES (Amazon)

Q81. Tell me about a time you improved a metric significantly.

Structure: STAR (Situation → Task → Action → Result). Quantify: "improved CTR by 15%" is better than "improved CTR." For Amazon: tie to Leadership Principles (Customer Obsession, Dive Deep, Deliver Results). Template: "We had [metric/problem]. I identified [root cause]. I [actions taken: algorithm, experiment design]. The result was [quantified impact]. I also [what I learned or changed as a result of this work]."

Q82. How do you handle a disagreement with a stakeholder about which metric to optimize?

Amazon LP: "Have Backbone; Disagree and Commit." Approach: (1) Understand their perspective fully — what is their constraint/concern? (2) Articulate your view with data — why does your metric better capture long-term value? (3) Propose experiment to compare both. (4) If overruled, commit fully to the decision. Key: distinguish between "disagree with the decision" and "disagree with the process." Data and customer impact should drive the resolution.

Q83. Describe a time you had to work with ambiguous requirements.

Amazon LP: "Bias for Action," "Invent and Simplify." Approach: (1) Define a clear, answerable version of the problem. (2) Identify the simplest experiment to reduce uncertainty. (3) Communicate assumptions explicitly to stakeholders. (4) Iterate rapidly. Show: you do not need perfect information to start, but you manage risk by validating assumptions early and cheaply.

Q84. Tell me about a machine learning model you built that had business impact.

Structure: business problem → data strategy → model choice → key challenges → deployment → measurable impact. Depth expected for Applied Scientist: explain your model choice (why GBT over LR?), what features were most important, how you evaluated offline vs online, what went wrong in production and how you fixed it. Amazon cares about: Customer Obsession, Dive Deep, Deliver Results.

Q85. How do you prioritize between multiple ML projects?

Framework: (1) Customer impact: which has highest expected value for customers? (2) Effort vs. impact: quick wins that unblock others vs. long-term bets. (3) Dependencies: what is blocking other teams? (4) Risk: what is the uncertainty? Can we de-risk with a cheap experiment? For Amazon: frame through Leadership Principles — Customer Obsession, Think Big, Deliver Results. Be specific about how you make the tradeoff quantitatively.

ADDITIONAL DEPTH QUESTIONS

Q86. What is the difference between frequentist CI and Bayesian credible interval?

Frequentist 95% CI: "If we repeated this experiment many times, 95% of constructed intervals would contain the true parameter." The parameter is fixed, not random. Bayesian 95% CrI: "Given data, $P(\theta \in [a,b]) = 0.95$." Parameter is random (has a distribution). This is the intuitive interpretation most people want. Requires a prior. With large data, both usually agree numerically.

Q87. Explain the multiple testing problem.

If you test 20 independent hypotheses at $\alpha=0.05$, you expect 1 false positive by chance. Family-wise error rate (FWER): Bonferroni correction — use α/m per test. Conservative. False discovery rate (FDR): Benjamini-Hochberg procedure — control the expected proportion of false positives among all rejections.

Less conservative. Practical: pre-register primary metric, treat secondary as exploratory. Use BH-corrected p-values when testing many metrics.

Q88. What is Simpson's Paradox?

A trend appears in several groups but reverses when groups are combined. Example: Treatment A appears better than B in both men and women separately, but appears worse in the combined dataset — because treatment assignment is correlated with gender, and gender affects outcomes. Cause: confounding variable correlated with both treatment and outcome. Lesson: always stratify analysis by important confounders. Draw the causal DAG — it immediately reveals the issue.

Q89. Explain bootstrapping. When is it useful?

Resample with replacement B times (B=1000). Compute statistic each time. CI from [2.5th, 97.5th percentile] of bootstrap distribution. Use when: no closed-form CI formula exists, data is non-normal, small sample size, complex statistics (correlation of correlations, custom metrics). Assumes: original sample is representative of population. BCa (bias-corrected accelerated) bootstrap handles skewed distributions better.

Q90. What is the Expectation-Maximization algorithm? Give an example.

EM: optimize models with hidden/latent variables by alternating: E-step (compute expected values of latents given current θ) and M-step (update θ to maximize expected complete-data log-likelihood). Monotonically increases log-likelihood. Converges to local maximum. Example — GMM: E-step: compute soft cluster assignments r■■ = P(cluster k | x■, θ). M-step: update cluster means μ■ = Σ■ r■■ x■ / Σ■ r■■ and covariances.

Q91. How does stochastic depth (DropPath) work?

During training, randomly drop entire residual blocks with probability p. Block i has survival probability: p■ = 1 - (i/L) × (1 - p_final). Deeper layers dropped more often. Interpretation: ensemble of networks with different depths. Benefits: strong regularization for deep networks, faster training. Used extensively in Vision Transformers (ViT), EfficientNets. Complement to dropout (which drops individual neurons).

Q92. What is Federated Learning?

Train ML models across decentralized data without centralizing it. Each device trains locally, sends only model updates (gradients) to server. Server aggregates updates (FedAvg: weighted average of local models). Privacy benefit: raw data never leaves the device. Challenges: non-IID data across devices, communication efficiency, device heterogeneity, stragglers. Combined with differential privacy: add calibrated noise to updates to prevent reverse-engineering individual data.

Q93. What is the difference between hard and soft parameter sharing in multi-task learning?

Hard sharing: all tasks share the same hidden layers, task-specific output heads. Implicit regularizer — reduces overfitting. Soft sharing: each task has its own network, but parameters are regularized to be similar. (MTL with L2 distance between task-specific weights.) More flexible but more parameters. Modern approach: use a shared pre-trained backbone (BERT, ViT) with task-specific adapters — best of both worlds.

Q94. Explain gradient accumulation and why it is used.

Accumulate gradients over multiple mini-batches before updating weights. Effective batch size = mini_batch_size × accumulation_steps. Why: training with very large batch sizes improves training stability, but VRAM cannot hold a large batch. Implementation: compute loss, call backward() for N steps without optimizer.step(). Then step() and zero_grad(). Equivalent to large batch training when using fixed learning

rate. Linear scaling rule: lr ∝ batch_size.

Q95. What is the vanishing gradient problem in RNNs? How do LSTMs solve it?

RNNs: hidden state h_t = tanh(W_h h_{t-1} + W_x x_t). Gradient w.r.t. h_0 = product of Jacobians across T steps. If eigenvalues of W_h < 1: exponential decay (vanishing). If > 1: exponential growth (exploding). LSTM solution: cell state c_t provides a "gradient highway" with additive update. Forget gate controls what to erase. Input gate controls what to add. Gradient flows through c without multiplication by W_h at each step.

Q96. What is self-supervised learning? Give examples.

Create supervision signals from the data itself — no human labels needed. Examples: Masked LM (predict 15% masked tokens — BERT), Next token prediction (GPT), Contrastive (SimCLR, CLIP), Rotation prediction (predict image rotation angle), Colorization (predict color from grayscale). Why powerful: leverage vast unlabeled data. Rich supervision signal emerges from hard prediction tasks. Pre-trained representations generalize better than supervised-only.

Q97. What is NDCG? How do you calculate it?

Normalized Discounted Cumulative Gain — standard ranking metric. CG = Σ rel■ (no position discount). DCG = Σ■■■■ (2^rel■ - 1) / log■(i+1) — higher positions get less discount. IDCG = DCG of ideal (perfectly sorted) ranking. NDCG = DCG / IDCG ∈ [0, 1], 1 = perfect ranking. Graded relevance: 0=irrelevant, 1=marginally, 2=relevant, 3=highly relevant. Average over query set for final NDCG@k score.

Q98. What is the difference between recall-oriented and precision-oriented retrieval?

Recall-oriented: maximize fraction of relevant items retrieved. Use case: legal e-discovery (find ALL relevant documents), medical research (missing a study is costly). Accept many false positives. Precision-oriented: maximize fraction of retrieved items that are relevant. Use case: search engine top-10 results, recommendation widget (user sees only 10 items — make them count). Tradeoff is inherent: you cannot maximize both without perfect retrieval. AUC-PR captures the full tradeoff curve.

Q99. What is the difference between homoscedasticity and heteroscedasticity?

Homoscedasticity: residual variance is constant across all values of X. Assumption of standard OLS regression. Heteroscedasticity: residual variance changes with X. Example: income prediction — errors are larger for high-income people. Consequence: OLS estimates are unbiased but inefficient; standard errors are wrong. Detection: Breusch-Pagan test, plot residuals vs fitted values. Fix: WLS (weighted least squares), log-transform target, robust standard errors.

Q100. Explain the two-tower architecture for large-scale retrieval. Why is it preferred?

Two-tower (bi-encoder): encode user and item independently into embeddings. Score: dot product or cosine similarity. Why preferred for retrieval: (1) Pre-compute item embeddings offline. (2) At query time: encode user, run ANN search over pre-computed item embeddings. (3) Scales to billions of items with <100ms latency via FAISS. Cross-encoder (full attention over [user, item] pair): much more accurate but O(n) per query — not feasible for retrieval. Use cross-encoder only for re-ranking top-100 candidates.

You've Got This

This document covers everything you need to ace Applied Scientist interviews at Amazon and Microsoft. Review one chapter per day, implement the algorithms, and practice explaining concepts out loud. Good luck!

ML Foundations · Core Algorithms · Deep Learning · Transformers & LLMs · Statistics · A/B Testing · System Design · Coding · Top 100 Q&A;